

Bachelorarbeit

Visualisierung kinematischer Kennwerte

An der Fachhochschule Dortmund
im Fachbereich Informatik
Studiengang BA Informatik
6. Fachsemester

von

Philipp Jan Mikos

geb. am 18.02.1993

Matr.-Nr. 7215204

Betreuer: Prof. Dr. Christof Röhrig

Dortmund, 8. August 2025

Inhaltsverzeichnis

1	Einleitung	1
2	Theoretischer Hintergrund	2
2.1	Freiheitsgrad	2
2.2	Industrieroboter	3
2.3	Vorwärtstransformation	6
2.4	Denavit-Hartenberg-Konvention	7
2.5	Rückwärtstransformation	9
2.6	Unified Robot Description Format (URDF)	10
3	Anforderungen	11
4	Entwicklungsprozess	13
4.1	Beschaffung von 3D-Modellen	13
4.2	Laden von 3D-Modellen	16
4.3	Graphische Darstellung von Industrierobotern	17
4.4	URDF-Parsing und Verarbeitung	20
4.5	Objektorientierte Modellierung der Kinematik	21
4.6	Animation	22
4.7	grafische Benutzeroberfläche	24
4.8	Denavit-Hartenberg-Konvention	27
4.9	Rückwärtstransformation (Inverse Kinematik)	30
5	Aufbau des Frameworks	35
5.1	Technischer Aufbau	35
5.2	Softwarearchitektur Model-View-Controller	36
5.2.1	Model-Komponente	36
5.2.2	View-Komponente	38
5.2.3	Controller-Komponente	38
6	Deployment des Frameworks	39

7	Evaluation	41
7.1	Stresstest zur Ressourcen- und Performance-Analyse	41
7.1.1	Testaufbau	41
7.1.2	Messmethoden	42
7.1.3	Durchführung	43
7.1.4	Ergebnisse	43
7.1.5	Interpretation	45
8	Anhang	46

Abbildungsverzeichnis

1	a) Quader mit Freiheitsgrad 3. b) Quader mit Freiheitsgrad 6. Quelle: (Weber und Koch (2022, S. 35))	2
2	Industrieroboter von Kuka. Quelle: (Weber und Koch (2022, S. 6)) . . .	4
3	Leichtbauroboter iiwa von KUKA. Quelle: (KUKA Contributors (2025))	5
4	schematischer Aufbau der Vorwärtstransformation. Quelle: (Weber und Koch (2022, S. 49))	7
5	schematische Zeichnung eines Knickarmroboters mit 6 Gelenken und DH-Koordinatensystemen. Quelle: (Weber und Koch (2022, S. 39)) . .	8
6	schematische Zeichnung eines Planarroboters mit 2 Achsen. Quelle: (Weber und Koch (2022, S. 51))	9
7	schematische Zeichnung einer Gelenkstellung die zu Unendlich vielen Lösungen führt. Quelle: (Weber und Koch (2022, S. 51))	10
8	Ansicht der 3D-Szene des Frameworks mit einzelnen Gliedern des KR6 R900-2 von KUKA	20
9	Ansicht der 3D-Szene des Frameworks mit dem KR6 R900-2 von KUKA in Nullstellung mit Greifer	22
10	Benutzeroberfläche von PolyScope5. Quelle: (Universal Robots (2025))	24
11	Viewport mit grafischer Benutzeroberfläche	26
12	Ansicht der 3D-Szene des Frameworks mit dem KR6 R900-2 von KUKA mit Denavit-Hartenberg-Koordinatensystemen	29
13	Verzeichnisstruktur des Packets visualkinematics_v2	35
14	Testaufbau der Ressourcen- und Performance-Analyse	42
15	Verteilung der gemessenen SPS-Werte je nach Anzahl der Clients . . .	44
16	UML-Klassendiagramm des Frameworks generiert mit dem Tool py2puml	48
17	UML-Packagediagramm der Module	49

Tabellenverzeichnis

1	Online-Quellen für 3D-Robotermodelle	14
2	Systemauslastung bei 4 und 8 JupyterLab-Instanzen	44

3	Technische Spezifikationen des Testsystems „System B LT“	46
4	Technische Spezifikationen des Testsystems „System C WS“	46
5	Technische Spezifikationen des Testsystems „System D AP“	47
6	Technische Spezifikationen des Testsystems „System A DT“	47
7	Technische Spezifikationen des Zielservers (JupyterHub-VM)	47

Algorithmusverzeichnis

1	Laden eines 3D-Modells aus einer NPZ-Datei	17
2	Berechnung von Vertex-Normalen	19
3	Gleichzeitige Interpolation mehrerer Gelenke	23
4	Iterative Inverse Kinematik mit symbolischem Vorwärtspfad und Gelenkwinkelgrenzen	33

1 Einleitung

Bei der Robotersimulation geht es darum das Verhalten eines Roboters in einer Softwareumgebung virtuell nachzubilden und somit das Testen und Programmieren des Roboters außerhalb der Produktivumgebung zu ermöglichen. Dabei werden häufig grafische Softwaresysteme verwendet, die die Umgebung und Bewegung des Roboters visuell darstellen. Gerade in der Offline-Programmierung spielt die Robotersimulation eine zentrale Rolle, um die textuelle Programmierung in Form von CAD-basierter Programmierung zu ergänzen. Auf diese Weise lässt sich das zeitaufwendige Teachen in der Produktionsumgebung deutlich reduzieren (Weber und Koch (2022, S. 113, 122)).

Im Rahmen dieser Arbeit wird ein Framework zur Robotersimulation entwickelt, das innerhalb der JupyterLab-Umgebung genutzt werden kann und in erster Linie für den Einsatz in der Lehre vorgesehen ist. Das Framework wird anschließend auf einem JupyterHub-Server als Kernel eingerichtet, sodass es Studierenden zur Verfügung steht.

Die vorliegende Arbeit baut auf den Ergebnissen einer vorausgegangenen Projektarbeit auf, in der verschiedene Grafikbibliotheken evaluiert sowie grundlegende Funktionalitäten wie die Erstellung einer 3D-Umgebung, Transformationen und Animationen von Körpern umgesetzt wurden. Auch die Simulation der Bewegung eines radgetriebenen Roboters wurde im Rahmen dieser Projektarbeit realisiert. Aufbauend auf diesen Ergebnissen wird das entwickelte Framework in dieser Arbeit um die Steuerung und Animation beliebiger Industrieroboter mit serieller Kinematik erweitert.

Ziel dieser Arbeit ist die Erweiterung bestehender Jupyter-Notebooks im Lehrbetrieb der Fachhochschule Dortmund um visuelle und interaktive Komponenten, insbesondere im technischen Modulfeld wie der Robotik. Durch die Integration anschaulicher Visualisierungen und direkter Benutzerinteraktionen soll das Verständnis kinematischer Zusammenhänge gefördert und die Lehre praxisnäher gestaltet werden.

2 Theoretischer Hintergrund

2.1 Freiheitsgrad

”Der Freiheitsgrad f eines Körpers ist die Anzahl der möglichen unabhängigen Bewegungen eines starren Körpers gegenüber einem Bezugskoordinatensystem”(Weber und Koch (2022, S. 35)). Dabei ist f die kleinste Menge von Parametern, die benötigt wird um die Lage des Körpers im Raum zu beschreiben (Weber und Koch (2022, S. 35)). Abbildung 1 zeigt jeweils einen Quader mit dem Freiheitsgrad 3 und dem Freiheitsgrad 6.

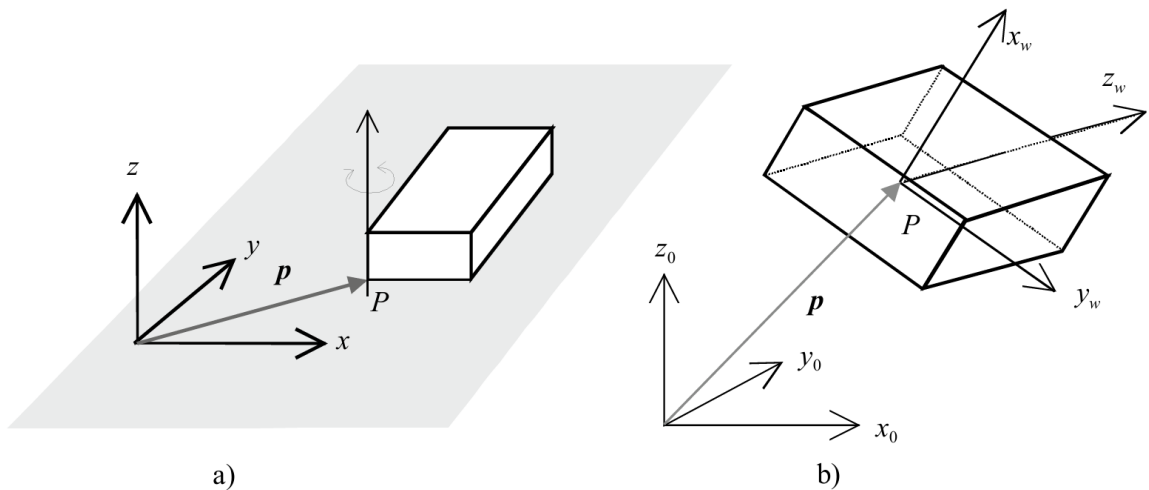


Abbildung 1: a) Quader mit Freiheitsgrad 3. b) Quader mit Freiheitsgrad 6. Quelle: (Weber und Koch (2022, S. 35))

Der Quader in Abbildung 1 a) hat den Freiheitsgrad 3 und kann nur auf einer Ebene bewegt werden die durch den grauen Bereich dargestellt wird. Die Position des Quaders wird durch einen Ortsvektor eindeutig beschrieben. Zusätzlich kann der Quader noch um eine Achse gedreht werden die Senkrecht zur Ebene steht und durch den Punkt P geht. Anhand des Ortsvektors und des Drehwinkels wird die Lage des Quaders auf der Ebene eindeutig beschrieben.

Der Quader in Abbildung 1 b) hat den Freiheitsgrad 6 und kann sich frei im Raum bewegen. Die Position eines Punktes P , für den sich der Schwerpunkt des Quaders eignet, wird durch einen Ortsvektor mit 3 Komponenten beschrieben. Das sind die 3

Freiheitsgrade der Translation. Hinzu kommen die 3 Freiheitsgrade der Rotation, da der Quader frei um 3 Achsen im Raum rotieren kann (Weber und Koch (2022, S. 35)).

In der Robotik bezeichnet der Arbeitsraum den Bereich im Raum, welcher vom Roboter erreicht werden kann. Ein Roboter der sich im Euklidischen Raum bewegt hätte den Arbeitsraum $T \subset \mathbb{R}^3 \times SO(3)$

Neben dem eben erwähnten Freiheitsgrad der sich auf den Arbeitsraum bezieht gibt es noch den Freiheitsgrad im Konfigurationsraum. In der Mechanik bezeichnet man die kleinste Menge von Parametern, die benötigt werden um die Lage eines Systems im Arbeitsraum zu beschreiben als Konfiguration eines Systems, die durch einen Punkt q dargestellt werden kann. Die Menge aller möglichen Konfigurationen bildet den sogenannten Konfigurationsraum (c-space) C , sodass $q \in C$. Ein Roboter mit 2 Drehgelenken hätte beispielsweise den Konfigurationsraum $C \subset SO(1) \times SO(1)$ und seine Konfiguration könnte anhand von zwei Parametern (q_1, q_2) beschrieben werden. Folglich hätte dieser Roboter einen Freiheitsgrad von 2 im Konfigurationsraum.

Jeder Punkt im Konfigurationsraum kann auf einen Punkt im Arbeitsraum abgebildet werden, wobei i.A nicht jeder Punkt im Arbeitsraum auf einen Punkt im Konfigurationsraum abgebildet werden kann. (Corke (2023, S. 78)).

2.2 Industrieroboter

Ein Industrieroboter (engl. industrial-robot, manipulator), ist gemäß DIN EN ISO 10218-1 ein frei programmierbarer Mehrzweck-Manipulator, der in 3 oder mehr Achsen programmierbar ist (Reinhart u. a. (2018, S. 17)). Er ist universell einsetzbar, und wird häufig für Schweiß-, -Lackierarbeiten, Montageaufgaben und viele weitere Fertigungs- und Handhabeaufgaben verwendet (Weber und Koch (2022, S. 2–3)). Obwohl ein Industrieroboter sich, im Gegensatz zu mobilen Robotern, nicht frei durch den Raum bewegen kann, kann er statt an festen Orten auch beweglich angeordnet werden (Reinhart u. a. (2018, S. 17)). Typischerweise besteht ein solcher Roboter aus mehreren starren Segmenten, sogenannten Gliedern (engl. links), welche durch Gelenke bzw. Achsen (engl. joints) miteinander verbunden sind. Der Industrieroboter führt seine Bewegung über seine Gelenke und Glieder aus, wobei ihre Anordnung die kinematische Struktur des Roboters bestimmt. Es gibt zwei Hauptklassen kinematischer Strukturen: Die serielle Kinematik und die Parallelkinematik. Die Parallelkinematik wird nur in spe-

zielleren Fällen verwendet wo eine besonders schnelle Handhabung notwendig ist und wird in dieser Arbeit nicht näher beleuchtet. Die serielle Kinematik ist in der Industrie häufiger vertreten. Die folgende Abbildung zeigt einen Industrieroboter von KUKA, welcher 6 Drehgelenke hat und zu den Seriellen Robotern gehört.

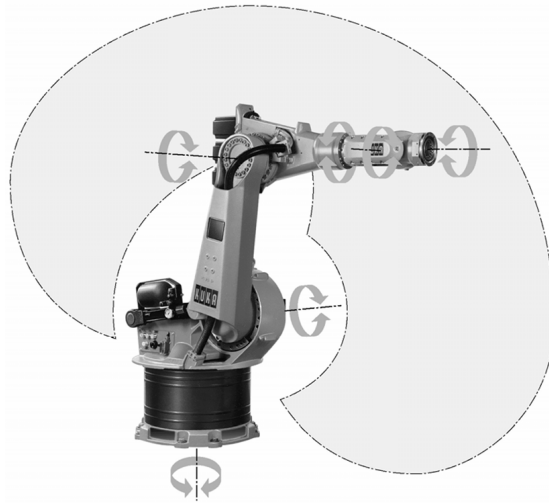


Abbildung 2: Industrieroboter von Kuka. Quelle: (Weber und Koch (2022, S. 6))

Die durch Pfeile markierten Elemente des Roboters sind die Drehgelenke. Durch Drehung dieser Gelenke werden die damit verbundenen Glieder im Raum bewegt. Die Glieder stehen aneinandergereiht zueinander und bilden eine kinematische Kette, was das Merkmal für Roboter mit serieller Kinematik ist (Weber und Koch (2022, S. 3–6)). Am oberen Ende des Roboters befindet sich ein Flansch, an den ein beliebiges Werkzeug, wie beispielsweise ein Greifer angebracht werden kann (Reinhart u. a. (2018, S. 17)). Ein solches Werkzeug wird dann als Endeffektor bezeichnet und kann als letztes Glied aufgefasst werden. Die Werkzeugspitze des Endeffektors wird als Tool Center Point (TCP) bezeichnet und wird verwendet um die sogenannte Pose (Position und Orientierung) des Endeffektors, im Raum zu beschreiben. Die Aufgabe eines jeden Industrieroboters ist es den Endeffektor geeignet im Raum zu bewegen um eine bestimmte Tätigkeit auszuführen. Dies wird durch eine passende Konfiguration der Achsen erreicht. Die Bewegungsmöglichkeit des Roboters hängt direkt von der Anzahl der Achsen ab. Man spricht bei der Anzahl unabhängig angetriebener Achsen vom mechanischen- oder Getriebe-Freiheitsgrad. Bei mindestens 6 unabhängig angetriebener Achsen in geeigneter Anordnung hat der Endeffektor den Freiheitsgrad 6 und

kann eine beliebige Pose einnehmen (Weber und Koch (2022, S. 3–6)). Der Bereich im Raum, der vom Endeffektor erreicht werden kann, wird Arbeitsraum des Roboters genannt und ist abhängig von der Bewegungsform, der Anordnung und Anzahl der Achsen sowie der Länge der einzelnen Glieder. Der Arbeitsraum eines Industrieroboters kann z.B. quaderförmig, zylindrisch oder kugelförmig sein. Der Arbeitsraum eines klassischen Sechs-Achs-Knickarmroboters ist kugelförmig wie in Abbildung 2 durch den grauen Bereich dargestellt ist. (Reinhart u. a. (2018, S. 18)). Es gibt auch Roboter mit mehr Achsen und einem höheren Freiheitsgrad, sowie weniger Achsen und einem geringeren Freiheitsgrad. Die Kinematik bei Robotern mit einem Freiheitsgrad > 6 ist redundant und gilt als überbestimmt (Weber und Koch (2022, S. 4–5)). Abbildung 3 zeigt den Leichtbauroboter iiwa von KUKA mit 7 Drehachsen.



Abbildung 3: Leichtbauroboter iiwa von KUKA. Quelle: (KUKA Contributors (2025))

Der Vorteil der überbestimmten Kinematik liegt darin, dass die Feinbewegung des Endeffektors verbessert wird und dieser besser mit bewegenden Objekten synchronisiert werden kann. Darüber hinaus sind Roboter mit 7 Drehachsen in Zusammenarbeit mit Menschen zu bevorzugen, da sie Hindernisse besser umfahren und sich somit der Bewegung von Menschen besser anpassen können (Weingartshofer u. a. (2019)).

Damit ein Industrieroboter durch Bewegung eine spezifische Tätigkeit ausführen

kann, muss er zunächst programmiert werden. Dabei "[...] wird in der Robotik zwischen zwei wesentlichen Verfahrensgruppen unterschieden: Online- und Offline-Programmierung"(Reinhart u. a. (2018, S. 149)). Die Online-Programmierung wird meistens durch sogenannte Teach-In verfahren durchgeführt. Dabei steuert eine Person den Roboter mithilfe eines Programmierhandgeräts an bestimmte Punkte im Raum. Anschließend wird diese Position gespeichert, sodass sie in Zukunft ohne erneute Steuerung angefahren werden kann. Die Offline-Programmierung umfasst die Verfahren zur Programmierung des Roboters ohne diesen einzubinden und eignet sich besonders wenn der Roboter komplexe Bahnen verfahren muss (Reinhart u. a. (2018, S. 149–150)). Dabei stellt die textuelle Programmierung, bei der die "[...] Befehle in eine Textdatei geschrieben werden und [...] sequentiell von der Robotersteuerung abgeabeitet [...]"werden, das am häufigsten verwendete Verfahren dar (Reinhart u. a. (2018, S. 149)).

In der heutigen Zeit findet die simulationsgestützte Programmierung, welche ebenfalls zu den Offline-Verfahren gehört, immer mehr Anwendung. Schon innerhalb des Produktionsprozesses sämtlicher Roboter-Komponenten werden CAD-Daten erstellt, die ein Digitales Modell dieser Komponenten darstellen. Mithilfe dieser CAD-Daten kann innerhalb von simulationsgestützten Offline-Tools ein Roboter dargestellt und simuliert werden. Diese Art der Roboter-Programmierung birgt viele Vorteile wie das Testen auf Kollision ohne Gefahr oder die Zeit und Kostenberechnung des Robotereinsatzes in der Planungsphase. Mit der Erstellung dieser simulationsgestützten Offline-Tools geht allerdings ein hohes Expertenwissen im bereich der Robotik einher (Reinhart u. a. (2018, S. 150–152)).

2.3 Vorwärtstransformation

Die Vorwärtstransformation, auch direktes Kinematisches Problem genannt, ist ein zentrales Konzept für die Steuerung und Simulation von Industrierobotern. Sie ist eine eindeutige Abbildung von Gelenkkoordinaten im Konfigurationsraum auf kartesische Koordinaten im Arbeitsraum. Die Pose des Roboters im Arbeitsraum wird vollständig durch die Gelenkkoordinaten beschrieben. Insbesondere ist dann auch die Pose des Endeffektors eindeutig festgelegt. Die folgende Abbildung zeigt den schematischen Ablauf der Vorwärtstransformation.

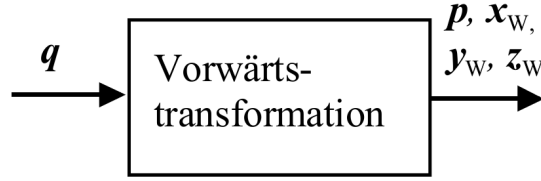


Abbildung 4: schematischer Aufbau der Vorwärtstransformation. Quelle: (Weber und Koch (2022, S. 49))

Sei q ein Tupel mit $q = (q_1, q_2, \dots, q_n)^T$, welches die Gelenkkoordinaten enthält. Der Ortsvektor p , ausgehend vom Ursprung des Bezugskoordinatensystems und die Orientierung, beschrieben durch die Basisvektoren x_W, y_W, z_W beschreiben die Pose des Endeffektors. Durch die Vorwärtstransformation können diese aus dem Tupel q berechnet werden. Genauer betrachtet ist die Vorwärtstransformation die Multiplikation mehrerer Homogener Transformationsmatrizen, die jeweils die Lage eines einzelnen Gelenks im Bezug auf das vorherige Gelenk beschreiben.

$$T_W(q) = {}^nT(q) = {}^1_0T(q_1) \cdot {}^2_1T(q_2) \cdot \dots \cdot {}^{n-1}_{n-2}T(q_{n-1}) \cdot {}^n_{n-1}T(q_n)$$

Für die Berechnung der Homogenen Transformationsmatrizen wird in der Robotik die sogenannten Denavit-Hartenberg-Notation verwendet, aus der dann die Denavit-Hartenberg-Matrizen resultieren. (Weber und Koch (2022, S. 49–50)).

2.4 Denavit-Hartenberg-Konvention

Die Denavit-Hartenberg-Konvention wurde 1955 von Jacques Denavit und Richard Hartenberg eingeführt und ist ein weit verbreitetes Werkzeug zur Beschreibung der Pose eines Roboters im Raum. Sie ist ein standardisiertes Verfahren zur Modellierung der Kinematik serieller Roboter. Ziel der Konvention ist es, die Position und Orientierung jedes Gliedes relativ zum vorhergehenden Glied mit einem minimalen Satz von Parametern darzustellen. Die Transformation von einem Glied zum nächsten wird durch eine homogene Transformationsmatrix beschrieben, die vier aufeinanderfolgende Operationen zusammenfasst: zwei Rotationen und zwei Translationen. Daraus ergeben sich die vier Denavit-Hartenberg-Parameter (DH-Parameter):

θ_i : der Gelenkwinkel, Rotation um die z-Achse

d_i : Verschiebung entlang der z-Achse

a_i : Verschiebung entlang der x-Achse

α_i : Rotation um die x-Achse

Mit diesen Parametern lässt sich für jedes Glied eine Homogene Transformationsmatrix aufstellen die fest mit dem Glied verbunden ist und die Position und Orientierung in Bezug zum vorherigen Koordinatensystem beschreibt. Die folgende Abbildung zeigt eine schematische Zeichnung eines Knickarmroboters mit 6 Gelenken und entsprechenden Denavit-Hartenberg-Koordinatensystemen.

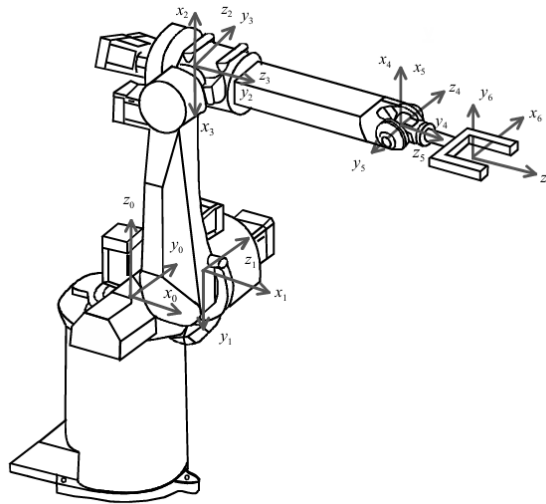


Abbildung 5: schematische Zeichnung eines Knickarmroboters mit 6 Gelenken und DH-Koordinatensystemen. Quelle: (Weber und Koch (2022, S. 39))

Die erste Transformationsmatrix beschreibt das Basiskoordinatensystem K_0 und ist fest mit der ruhenden Basis, also dem ersten Glied verbunden. Der Ursprung von K_0 muss dabei auf die erste Gelenkachse gelegt werden. Die z_0 -Achse von K_0 zeigt entlang der ersten Gelenkachse. Die x_0 - und y_0 -Achse sind anschließend frei wählbar unter der bedingung das die zugehörigen Einheitsvektoren x_0 , y_0 und z_0 ein Rechtssystem bilden.

Jedes weitere Koordinatensystem K_i , welches durch eine Transformationsmatrix repräsentiert wird, das aus den DH-Parametern gebildet wird, muss einen Satz fest definierter Bedingungen erfüllen: Der Ursprung von K_i sollte auf der Gelenkachse $i + 1$ liegen. Wenn sich die Gelenkachsen i und $i + 1$ schneiden, muss der Ursprung von K_i im Schnittpunkt dieser beiden Achsen liegen. Verlaufen die beiden Achsen jedoch parallel,

kann der Ursprung von K_i beliebig auf die Gelenkachse $i + 1$ gelegt werden. Wenn die Achsen sich nicht schneiden und auch nicht parallel sind wird die gemeinsame Normale der Gelenkachse i und $i + 1$ berechnet und der Ursprung von K_i auf den Schnittpunkt der gemeinsamen Normalen mit der Gelenkachse $i + 1$ gelegt.

Die Gesamttransformation vom Basis-Koordinatensystem K_0 zum Koordinatensystem des Endeffektors K_i ergibt sich durch sukzessive Multiplikation aller einzelnen Transformationsmatrizen $K_0, K_1, \dots, K_n$ entlang der kinematischen Kette. Das entspricht der Durchführung der Vorwärtstransformation. (Weber und Koch (2022, S. 38–44))

2.5 Rückwärtstransformation

Die Rückwärtstransformation, auch als inverse Kinematik bezeichnet, wird verwendet um aus der gegebenen Pose des Endeffektors die Gelenkkoordinaten $q_0, q_1, \dots, q_n$ zu berechnen und stellt somit das Gegenstück zu Vorwärtstransformation dar. Im Gegensatz zur Vorwärtstransformation bringt die Rückwärtstransformation einige Schwierigkeiten wie Mehrdeutigkeiten und Singularitäten mit sich. Eine Pose des Endeffektors innerhalb des Arbeitsraums kann unter Umständen mit mehreren verschiedenen Gelenkstellungen des Roboters erreicht werden. Die folgende Abbildung veranschaulicht eben diesen Umstand.

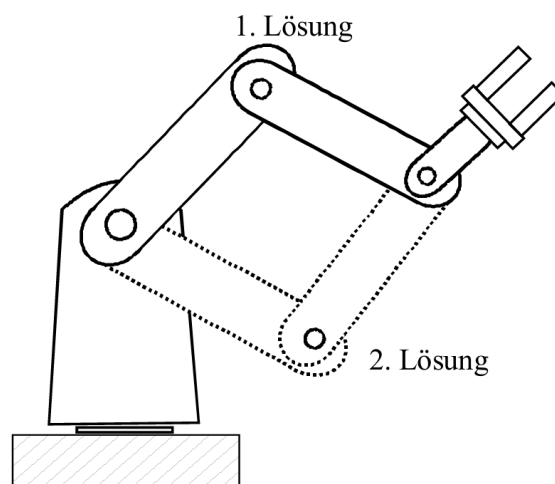


Abbildung 6: schematische Zeichnung eines Planarroboters mit 2 Achsen. Quelle: (Weber und Koch (2022, S. 51))

Unendlich viele Lösungen gibt es z.B dann, wenn der Roboter eine Position einnimmt, in der er einen Freiheitsgrad verliert und dadurch gleiche Änderungen an bestimmten Gelenken zu gleichen Änderungen am Endeffektor führen. Die folgende Abbildung veranschaulicht das Auftreten einer solchen Singularität.

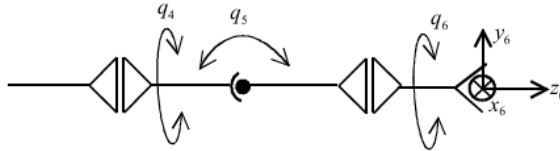


Abbildung 7: schematische Zeichnung einer Gelenkstellung die zu Unendlich vielen Lösungen führt. Quelle: (Weber und Koch (2022, S. 51))

In der vorliegenden Gelenkstellung von q_5 gibt es zu jeder Gelenkstellung von q_4 eine Gelenkstellung von q_6 die den Endeffektor in der gegebenen Pose beschreibt. Daraus folgen unendlich viele Konfigurationen die für den Roboter möglich wären um die gegebene Pose des Endeffektors zu erreichen. Um solche Singularitäten aufzulösen kann ein Gütekriterium als Nebenbedingung formuliert werden, um eine definierte Lösung zu erhalten.

Für die gängigen Knickarmroboter mit 6 Gelenken kann das geometrische verfahren angewendet werden um aus der Endeffektor Pose die Gelenkkoordinaten zu erhalten. Diese Methode wird in der Praxis bevorzugt angewendet. Für viele andere Roboter, wie beispielsweise Knickarmroboter mit mehr als 6 Gelenken kann das geometrische Verfahren nicht angewendet werden. In solchen Fällen werden Numerische Verfahren wie zum Beispiel das Newton-Raphson-Verfahren verwendet um eine geeignete Näherungslösung zu approximieren (Weber und Koch (2022, S. 49–56)). Da in dieser Arbeit das numerische Verfahren mittels Newton-Raphson-Verfahren dem geometrischen Verfahren vorgezogen wurde, wird dieses in dieser Arbeit noch eine ausführlichere Rolle spielen.

2.6 Unified Robot Description Format (URDF)

URDF ist ein portables, XML-basiertes Dateiformat, welches den Austausch von Roboter-Modellen erlaubt. Es erleichtert das einbinden von Roboter-Modellen in Simulationssoftware und ist in der Robotik sehr verbreitet. URDF-Dateien werden oft mit Xacro, einer XML macro Sprache erstellt um die Dateien kürzer und besser lesbar

zu machen. URDF-Dateien enthalten viele Informationen wie die Orientierung und Position von Gelenken und Gliedern, den Pfad zu den 3D-Modellen, welche die Glieder repräsentieren, Hierarchien innerhalb der kinematischen Kette sowie Minimal- und Maximal-Grenze der Gelenke und viele weitere Technische Informationen. Es existieren viele URDF-Dateien zu bekannten Industrierobotern die aus dem Internet von Seiten wie Github frei Runtergeladen und verwendet werden können (Corke (2023, S. 270–271)).

3 Anforderungen

Im Rahmen dieser Bachelorarbeit wird das in der vorangegangenen Projektarbeit entwickelte Framework erweitert. Während in der Projektarbeit der Fokus auf grundlegenden Transformationen und der Visualisierung kinematischer Konzepte lag, steht nun die Integration eines stationären Robotermodells, konkret eines Knickarmroboters, im Zentrum der Weiterentwicklung. Die folgenden funktionalen und nicht-funktionalen Anforderungen ergeben sich aus den Zielsetzungen dieser Arbeit:

- **Funktionale Anforderungen:**

- **Integration eines Knickarmroboters:**

Das Framework soll um ein Modell eines typischen Industrieroboters mit Knickarmstruktur erweitert werden. Dieses Modell bildet die Grundlage für die Umsetzung kinematischer Verfahren.

- **Vorwärts- und Rückwärtstransformation:**

Das Denavit-Hartenberg Modell des Roboters soll sowohl die Berechnung der Endeffektorposition aus gegebenen Gelenkwinkeln (Vorwärtstransformation) als auch die Bestimmung notwendiger Gelenkkonfigurationen für eine gegebene Endeffektorposition (Rückwärtstransformation) ermöglichen. Beide Transformationen sollen visuell nachvollziehbar simuliert und dargestellt werden.

- **Endeffektorsteuerung durch interaktive Bewegung des Koordinatensystems:**

Es soll möglich sein, das Koordinatensystem des Tool-Center-Point (TCP)

im dreidimensionalen Raum interaktiv zu verschieben und zu rotieren. Der Roboter soll dieser Bewegung durch inverse Kinematik folgen, wie es in gängigen Robotersteuerungen (z.B. bei Universal Robots) üblich ist.

– **Bewegung im lokalen und globalen Koordinatensystem:**

Die Steuerung des TCP soll sowohl im lokalen Werkzeugkoordinatensystem als auch im globalen Weltkoordinatensystem erfolgen können. Eine Umschaltung zwischen diesen Modi soll benutzerfreundlich möglich sein.

– **Manuelle Konfiguration der Gelenkwinkel:**

Über interaktive Schieberegler (Slider) soll der Nutzer die Gelenkwinkel des Roboters direkt beeinflussen können. Änderungen der Konfiguration sollen unmittelbar in der Visualisierung sichtbar werden, wobei die aktuelle Position des Endeffektors kontinuierlich anhand der Vorwärtstransformation berechnet wird.

• **Nicht-Funktionale Anforderungen:**

– **Effiziente Ausführung:**

Die Anwendung soll unter typischen Systemvoraussetzungen eine flüssige Benutzerinteraktion ohne spürbare Ruckler ermöglichen. Die 3D-Szene soll unter typischen Systemvoraussetzungen mit einer Bildwiederholrate von mindestens 60 Bildern pro Sekunde (FPS) gerendert werden. Die Aktualisierung der Roboterpose muss unter typischen Systemvoraussetzungen mindestens mit einer Frequenz von 60Hz erfolgen, um eine flüssige Bewegung unabhängig von der tatsächlichen Bildwiederholrate im Browser sicherzustellen.

– **Benutzerfreundliche Bedienbarkeit:**

Die Benutzeroberfläche soll intuitiv und ohne vorherige Schulung bedienbar sein. Alle Hauptfunktionen sollen mit maximal drei Klicks erreichbar sein. Fehlbedienungen sollen durch visuelles Feedback und validierte Eingabefelder minimiert werden.

– **Intuitive Vermittlung kinematischer Konzepte:**

Die Anwendung soll eine intuitive Zuordnung zwischen grafischer Darstellung und mathematischem Modell (DH-Konvention) ermöglichen. Die Be-

nutzeroberfläche soll so gestaltet sein, dass die Bedienung des Roboters gleichzeitig als Lernprozess dient.

Diese Anforderungen bauen auf den bereits in der Projektarbeit definierten Zielen auf, insbesondere auf der dort formulierten langfristigen Perspektive, stationäre Roboterkinematik in das Framework zu integrieren. Im Fokus steht weiterhin die intuitive Vermittlung komplexer kinematischer Zusammenhänge durch interaktive und visuelle Darstellungen in der browserbasierten JupyterLab-Umgebung.

4 Entwicklungsprozess

hier sollte noch was darüber stehen dass die basis des Frameworks die in der projektarbeit entwickelt wurde fähig ist den pythreejs Renderer über die display funktion in ein jupyter notebook einzubinden und der viewport dann wie ein widget bei ausführung der zelle in das Notebook eingebunden wird.

4.1 Beschaffung von 3D-Modellen

Die bereits in der Projektarbeit entwickelte Basis des Frameworks erlaubt die Erstellung von geometrischen Formen (Mikos (2025, S. 15)). Dafür wird im Backend die Rendering-API pythreejs verwendet. Diese bietet anhand von vorgefertigten Klassen wie BoxGeometry und CylinderGeometry eine einfache Möglichkeit primitive Geometrien zu erstellen und zu rendern (pythreejs Contributors (2025b)). Für die Erstellung von komplexen und individuellen Geometrien, wie die eines Industrieroboters oder dessen Glieder, bietet pythreejs die Klasse BufferGeometry, welche explizit mit den Vertices, Indices und Normals eines Mesh-Modells initialisiert werden kann (pythreejs Contributors (2025a)). Diese Daten können entweder aus bestehenden 3D-Dateien (z.B. .stl, .obj, .dae) extrahiert oder durch eigene Verfahren erzeugt werden.

Im Framework soll nicht ein einziges Mesh-Modell des vollständigen Roboters geladen werden, da sich dessen Gelenke in diesem Fall nicht unabhängig voneinander animieren ließen. Stattdessen muss das 3D-Modell des Roboters aus mehreren separaten Meshes bestehen, die jeweils ein einzelnes Glied (Link) repräsentieren. Diese Glieder werden anschließend entsprechend ihrer Positionen und Transformationen innerhalb der kinematischen Kette zusammengesetzt.

Da es im Internet viele 3D-Modelle von Industrierobotern in Form solcher 3D-Dateien gibt, war der erste Schritt zur Erweiterung des Frameworks die Recherche nach geeigneten Quellen solcher Modelle. Die Recherche ergab folgende Ergebnisse:

Quelle	Beschreibung	Dateiformate
ROS-Industrial	Offizielle Roboterbeschreibungen für MoveIt/ROS	URDF, STL, DAE
GrabCAD	CAD-Bibliothek mit vielen industriellen Modellen	STL, STEP, SLDASM
Thingiverse	Plattform für 3D-druckbare Modelle	STL, OBJ
Onshape Public Library	Online-CAD-Modelle mit direktem Export	beliebig
Peter Corke Robotics Toolbox	Kinematik- und Modellbibliothek für Lehre und Forschung	URDF, STL

Tabelle 1: Online-Quellen für 3D-Robotermodelle

ROS-Industrial ist ein Open-Source-Projekt welches Module und Ressourcen bereitstellt die innerhalb des ROS-Ökosystems genutzt werden können. Dabei werden diese Ressourcen sowohl von der Community genutzt wie auch gewartet und sind komplett Quelloffen. Viele dieser Ressourcen können unter Github von ROS-Industrial heruntergeladen werden (ROS-Industrial Contributors (2025b)). Zu diesen Ressourcen zählen auch 3D-Modelle von Industrierobotern die unter dem Github-Topic moveit gefunden werden können. Dort finden sich zahlreiche Repositories, die jeweils die Roboterbeschreibungen eines bestimmten Herstellers enthalten (ROS-Industrial Contributors (2025a)). Diese Repositories sind in der Regel als eigenständige ROS-Packages organisiert und enthalten häufig weitere Unterpackages für die einzelnen Robotermodelle. Dies entspricht der typischen Struktur innerhalb des ROS-Ökosystems, in dem Software, Konfigurationen und Modelle modular in Form von ROS-Packages aufgebaut sind (Open Source Robotics Foundation (2023)). Die Unterpackages enthalten zudem Dateien im Xacro-Format, wobei es sich um die URDF-Beschreibung des jeweiligen Modells handelt.

Eine weitere Quelle für 3D-Modelle von Industrierobotern ist GrabCAD, ein Unternehmen, das Softwarelösungen im Bereich 3D-Druck anbietet. Darüber hinaus betreibt GrabCAD eine Online-Plattform, auf der Mitglieder der Community CAD-Dateien hochladen, teilen und herunterladen können (GrabCAD Contributors (2025)). Auf

der Online-Plattform sind CAD-Dateien von Industrierobotern wie zum Beispiel der `kr_140_3200_2` von KUKA, der PUMA 560 und viele andere. Die CAD-Dateien liegen in den meisten Fällen im STEP-, STL- oder im SLDASM-Format vor und lassen sich mit CAD-Software wie beispielsweise FreeCAD öffnen und bearbeiten.

Weiterhin ist Thingiverse eine Quelle für 3D-Modelle von Industrierobotern. Ähnlich wie bei der Online-Plattform von GrabCAD handelt es sich bei Thingiverse ebenfalls um eine solche Plattform die von Ultimaker betrieben wird. Auch bei dieser Plattform gibt es zahlreiche CAD-Dateien von Industrierobotern die von der Community geteilt werden, wobei der Fokus eher auf dem 3D-Druck liegt (UltiMaker (2025)).

Bei Onshape handelt es sich um eine browserbasierte CAD-Software, die als browserbasierte Software-as-a-Service Lösung von dem Unternehmen PTC bereitgestellt wird. Als Nutzer der Plattform ist es möglich Repositories in der Cloud anzulegen und diese mit anderen Nutzern zu teilen (PTC (2025)). Hier befinden sich zahlreiche Repositories aus denen 3D-Modelle von Industrierobotern in einem beliebigen Format heruntergeladen werden können.

Abschließend wurde versucht die 3D-Modelle aus der Robotics-Toolbox von Peter Corke zu extrahieren, welche von Github oder über pip heruntergeladen und installiert werden kann. Die Toolbox bietet zahlreiche Funktionen für die Modellierung und Analyse robotischer Systeme und erlaubt die visuelle Simulation von Robotern innerhalb eines eigenen Renderers namens Swift (Corke (2023, S. 77)). Die dafür benötigten 3D-Modelle werden bei der Installation der Toolbox mitgeliefert und befinden sich samt einer URDF-Beschreibung im Verzeichnis `./robotics-toolbox-python/rtbdata/rtbdata/xacro`.

Die genannten Quellen stellen eine geeignete Möglichkeit zur Beschaffung der 3D-Modelle dar. Da es innerhalb des Frameworks auch möglich sein soll die Bewegung der Roboter zu simulieren, werden weitere Informationen wie die Position und Orientierung von Gelenken und Gliedern, Hierarchien innerhalb der kinematischen Kette sowie Minimal- und Maximal-Grenze der Gelenke benötigt. Diese Informationen liegen üblicherweise im URDF-Format vor. Obwohl es grundsätzlich möglich ist, diese Informationen manuell zu rekonstruieren und in das URDF-Format zu übertragen, ist dies mit erheblichem Aufwand verbunden. Daher sind Quellen, die neben den 3D-Modellen bereits eine vollständige URDF-Beschreibung bereitstellen, von besonderem Vorteil.

von den genannten Quellen stellen nur ROS-Industrial und die Robotics-Toolbox von Peter Corke solche URDF-Beschreibungen zur Verfügung. Darüber hinaus ist die Anzahl der Dreiecke in den Meshes dieser beiden Quellen bereits reduziert, was die Ladezeit verbessert. Aus diesen Gründen sind im weiteren Verlauf des Entwicklungsprozesses hauptsächlich diese beiden Quellen verwendet worden.

4.2 Laden von 3D-Modellen

Um die 3D-Modelle der Industrieroboter innerhalb des Frameworks rendern zu können, müssen diese zunächst aus den heruntergeladenen Dateien in den Arbeitsspeicher geladen werden. Hierfür wird die Python-Bibliothek `trimesh` verwendet, die das Einlesen und die Verarbeitung von triangulären Meshes unterstützt. Die Bibliothek erlaubt das Laden gängiger Formate wie STL, OBJ, DAE und weiterer (`trimesh Contributors (2025)`). Die geladenen Roboter bestehen dabei nicht aus einer einzigen Mesh, sondern setzen sich aus mehreren einzelnen Teil-Meshes zusammen, von denen jede durch eine eigene Datei (`.stl`, `.obj`, `.dae`) repräsentiert wird. Diese Teilmodelle entsprechen den physischen Gliedern des Roboters. Beim Laden der Meshes ist aufgefallen dass der Vorgang pro Mesh einige Sekunden in Anspruch nimmt und Insbesondere bei komplexeren Modellen zu einer Gesamtladezeit von knapp einer Minute pro Roboter-Modell geführt hat. Verwendet wurde das in Tabelle 6 dargestellte System A DT, wobei es sich um einen Desktop PC handelt. Um die Ladezeiten der Meshes zu reduzieren und dadurch die Benutzerfreundlichkeit zu verbessern, wurde das NPZ-Format von NumPy verwendet. Im Gegensatz zu den üblichen Text-basierten Formaten handelt es sich beim NPZ-Format um ein binäres Format, welches maschinell effizienter eingelesen werden kann (`NumPy Contributors (2025)`). Die 3D-Modelle wurden in das NPZ-Format konvertiert, um die Ladezeiten zu optimieren. Um die Auswirkung auf die Ladezeit nachzuvollziehen wurde eine Messung durchgeführt. Dabei wurde die Ladezeit beim Einlesen einer `.npz`-Datei mit derjenigen beim Laden der entsprechenden `.stl`-Datei für dieselbe Mesh verglichen. Der Test, welcher dieser Arbeit unter dem Namen `benchmark_npz_vs_stl.ipynb` beiliegt ergab, dass der Ladevorgang aus der `.stl` Datei c.a 0.7757 Sekunden benötigte. Der Ladevorgang aus der `.npz`-Datei benötigte hingegen c.a 0.0002 Sekunden und ist damit c.a 2800 mal schneller:

Ausführungszeit-STL: 0.775746 Sekunden für 570279 Faces

Ausführungszeit-NPZ: 0.000276 Sekunden für 570279 Faces

Für diesen Test ist das bereits erwähnte System A DT aus Tabelle 6 zum Einsatz gekommen. Um das NPZ-Format nutzen zu können, müssen die 3D-Modelle zunächst in dieses Format konvertiert und dauerhaft gespeichert werden. So können sie beim nächsten Start des Frameworks direkt aus der .npz-Datei geladen werden. Dafür wurde ein Lazy-Loading-Mechanismus implementiert, der zunächst prüft, ob bereits eine .npz-Datei für die gewünschte Mesh vorhanden ist. Abhängig vom Ergebnis wird die Datei entweder geladen oder zunächst erstellt:

Algorithm 1 Laden eines 3D-Modells aus einer NPZ-Datei

```
1: Input: filepath, filepath_npz
2: if not is_file(filepath_npz) then
3:   to_npz(filepath, filepath_npz)
4: end if
5: return load_from_npz(filepath_npz)
```

4.3 Graphische Darstellung von Industrierobotern

Die geladenen Daten werden anschließend in BufferGeometry-Objekte überführt, wie sie von pythreejs benötigt werden, um als Bestandteil einer Szene gerendert werden zu können. Bereits in der vorangegangenen Projektarbeit wurde hierfür eine Klasse namens Environment entwickelt, die die Erstellung einer Szene einschließlich Beleuchtung, Kamera, Basiskoordinatensystem und Gitter kapselt. Darüber hinaus verwaltet sie alle Objekte, die über die display-Funktion im Jupyter Notebook gerendert werden sollen (Mikos (2025, S. 15)). Diese Objekte erben von der Klasse Object3D aus pythreejs und können ihrerseits weitere Object3D-Instanzen über ein Array namens Children enthalten. Transformationen werden dabei an die Children weitergegeben, wodurch Transformationshierarchien entstehen. Dieses Prinzip lässt sich gezielt für die visuelle Darstellung und Animation von Robotern mit serieller Kinematik nutzen: Jedes Glied der kinematischen Kette verwaltet dabei das nachfolgende Gelenk als Child, welches wiederum das nächste Glied als Child verwaltet. Auf diese Weise bewegen sich alle nachfolgenden Glieder korrekt mit, wenn ein vorheriges Glied transformiert wird.

Im STL-Format, welches trianguläre Mesh-Modelle beschreibt und in den genutzten Quellen häufig verwendet wird, werden nur die Normalenvektoren pro Fläche eines Dreiecks gespeichert (Hallmann u. a. (2019, S. 297)). Das MeshStandardMaterial, welches in diesem Framework für alle Geometrien verwendet wird verwendet physically based rendering (PBR), wobei das Beleuchtungsmodell pro Fragment (pro Pixel) und nicht pro Fläche angewendet wird (Three.js Contributors (2025)). Aus diesem Grund müssen die Vertex-Normalen manuell berechnet werden, die dann im Hintergrund von pythreejs baryzentrisch Interpoliert werden um einen Normalenvektor pro Pixel zu bestimmen (Mileff (2024, S. 9)). Für die Berechnung der Normalen ist ein gängiger Algorithmus implementiert worden. Dabei wird für jeden Vertex $v_i \in \{v_0, v_1, \dots, v_n\} \subset \mathbb{R}^3$ eine gemittelte Normale $n_i \in \mathbb{R}^3$ bestimmt, indem zunächst für jedes Dreieck $t_k = (\alpha_k, \beta_k, \gamma_k)$ die Flächennormale $n_{face}^{(k)}$ berechnet wird:

$$\begin{aligned} e_1^{(k)} &= v_{\beta_k} - v_{\alpha_k} \\ e_2^{(k)} &= v_{\gamma_k} - v_{\alpha_k} \\ n_{face}^{(k)} &= e_1^{(k)} \times e_2^{(k)} \end{aligned}$$

Die Flächennormalen werden aufsummiert und es ergibt sich für jeden v_i :

$$n_i := \sum_{k: v_i \in t_k} n_{face}^{(k)}$$

Zuletzt wird n_i mit $n_i \leftarrow \frac{n_i}{\|n_i\|}$ normiert, sofern $\|n_i\| \neq 0$.

Algorithm 2 Berechnung von Vertex-Normalen

```
1: Input: vertices ( $N \times 3$ ), indices ( $Z$ )
2: normals  $\leftarrow$  Nullmatrix der Form  $N \times 3$ 
3: triangles  $\leftarrow$  Umgeformt aus indices zu  $M \times 3$ 
4: for  $k = 0$  to  $M - 1$  do
5:    $v_0 \leftarrow \text{vertices}[\text{triangles}[k, 0]]$ 
6:    $v_1 \leftarrow \text{vertices}[\text{triangles}[k, 1]]$ 
7:    $v_2 \leftarrow \text{vertices}[\text{triangles}[k, 2]]$ 
8:    $e_1 \leftarrow v_1 - v_0$ 
9:    $e_2 \leftarrow v_2 - v_0$ 
10:   $n_{\text{face}}^{(k)} \leftarrow e_1 \times e_2$ 
11:  for  $i \in \{0, 1, 2\}$  do
12:     $\text{normals}[\text{triangles}[k, i]] += n_{\text{face}}^{(k)}$ 
13:  end for
14: end for
15: for  $i = 0$  to  $N - 1$  do
16:   if  $\|\text{normals}[i]\| = 0$  then
17:      $\text{normals}[i] \leftarrow [0, 0, 1]$  ▷ Standardnormale
18:   else
19:      $\text{normals}[i] \leftarrow \frac{\text{normals}[i]}{\|\text{normals}[i]\|}$ 
20:   end if
21: end for
22: Return normals
```

Die so berechneten Normalen müssen dem BufferGeometry-Objekt übergeben werden und können dann für das Shading pro Pixel verwendet werden. Durch diese Form des Shadings wirken die Oberflächen von Komplexen Geometrien realistisch und glatt (Three.js Contributors (2025)). Abbildung 8 zeigt die mithilfe des Frameworks gerenderten Einzelglieder des KR6 R900-2 von KUKA.

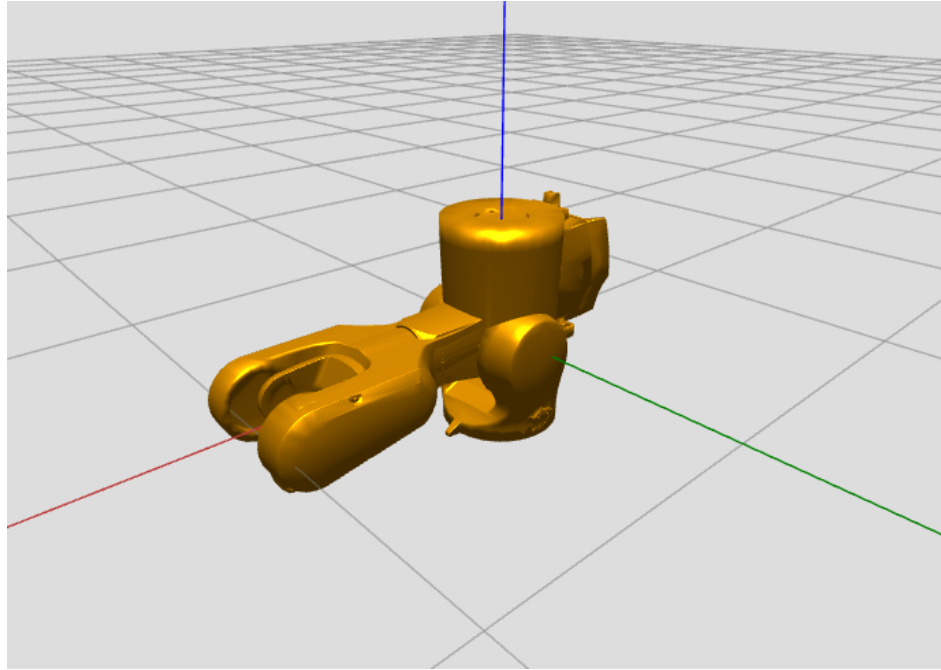


Abbildung 8: Ansicht der 3D-Szene des Frameworks mit einzelnen Gliedern des KR6 R900-2 von KUKA

In dieser Darstellung besteht noch keine Hierarchie zwischen den einzelnen Gliedern. Alle Glieder sind im Ursprung des Basiskoordinatensystems positioniert und bilden daher noch keine zusammenhängende Roboterstruktur. Die Farbe der Glieder ist frei gewählt worden.

4.4 URDF-Parsing und Verarbeitung

Um eine korrekte Darstellung zu erreichen, müssen die Glieder entsprechend einer URDF-Beschreibung räumlich korrekt positioniert und orientiert werden. Auch Farbinformationen der Glieder sowie der Pfad zum 3D-Modell lassen sich aus den URDF-Dateien der verwendeten Quellen extrahieren. Dazu ist es zunächst erforderlich, die vorhandenen .xacro-Dateien einzulesen und in gültiges URDF-Format zu überführen. Für diesen Zweck wird die Bibliothek xacrodoc verwendet, die über pip installiert werden kann. Mit Hilfe von xacrodoc lassen sich die Xacro-Dateien zur Laufzeit als URDF-String in den Arbeitsspeicher laden (Heins (2025)). Im weiteren Verlauf wurde eine Funktion implementiert, die diesen URDF-String analysiert und in ein strukturierteres Dictionary überführt, das innerhalb des Frameworks weiterverarbeitet werden kann um die Glieder und Gelenke des Roboters zu initialisieren. Um die Performance

zu steigern, werden die Glieder pro Robotermodell parallel verarbeitet, indem ihre Initialisierung jeweils in einem eigenen Thread erfolgt. Diese parallele Verarbeitung ist an dieser Stelle sinnvoll, da beim Initialisieren der Glieder auch das Laden der jeweiligen Meshes erfolgt. Dieses benötigt pro Mesh, wie in Kapitel 4.2 gezeigt, eine gewisse Zeit.

4.5 Objektorientierte Modellierung der Kinematik

Um den Code generisch zu halten und den allgemeinen Aufbau eines Industrieroboters abzubilden, wurden an dieser Stelle die Klassen Joint, Link, Tool und Manipulator erstellt. Die Klassen Link, Joint und Tool verwalten die relative Pose innerhalb der kinematischen Kette, sowie die Mesh und das Material. Die Klasse Joint verwaltet zusätzlich die relative Pose und die Minimal- und Maximalgrenzen sowie die Drehachse eines Gelenks. Da ein Joint keine Mesh hat, wurde hier als graphische Repräsentation lediglich die sogenannte Group Klasse von pythreejs genutzt, die keine Mesh beinhaltet, jedoch eine Pose im Raum besitzt, sodass trotz nicht vorhandener Geometrie dennoch eine Transformationshierarchie aufgestellt werden kann und die Transformationen an die Children weiter gegeben werden. Die 3 Klassen Link, Joint und Tool verwalten darüber hinaus ein Array von Children um die Hierarchie innerhalb der kinematischen Kette abzubilden. Die Klasse Manipulator repräsentiert den Industrieroboter und verwaltet Links, Joints und maximal eine Tool-Instanz. Der Konstruktor der Manipulator-Klasse ist so gestaltet worden, dass der Name des Robotermodells übergeben werden muss. Anschließend werden auf Basis des zuvor geparsen URDF-Strings automatisch die notwendigen Objekte wie Links und Joints erzeugt und miteinander verknüpft. Das Roboter-Objekt kann einer Environment-Instanz übergeben und somit gerendert werden. Abbildung 9 zeigt den mithilfe des Frameworks gerenderten KR6 R900-2 von KUKA in Nullstellung mit Greifer.

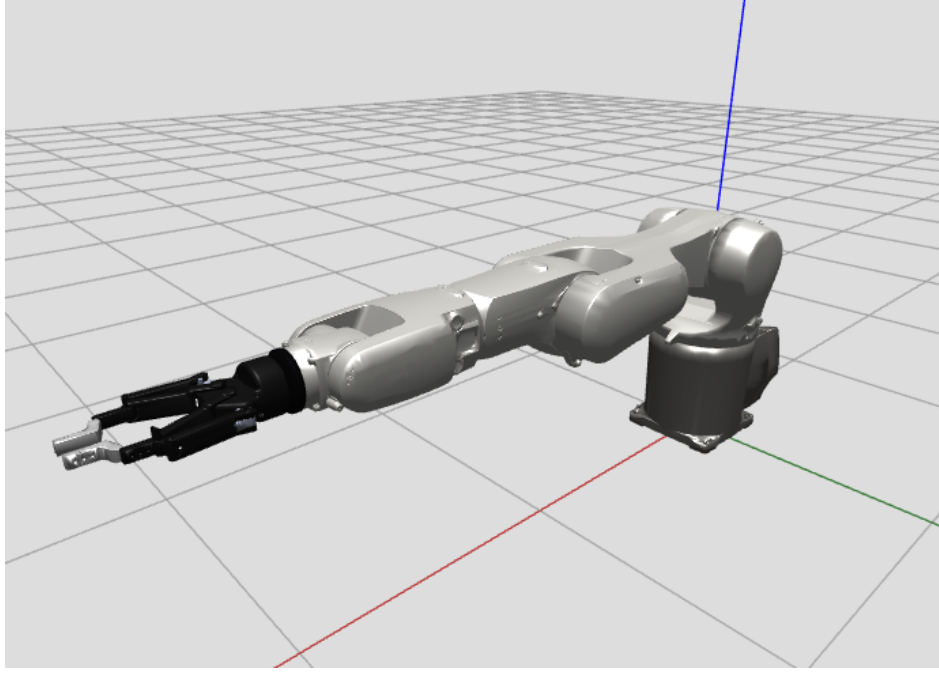


Abbildung 9: Ansicht der 3D-Szene des Frameworks mit dem KR6 R900-2 von KUKA in Nullstellung mit Greifer

Die einzelnen Glieder und Gelenke wurden dabei gemäß ihrer Beschreibung in der URDF-Datei korrekt transformiert und eingefärbt. So ergibt sich in ihrer Gesamtheit die vollständige Roboterstruktur.

4.6 Animation

Mithilfe des Frameworks soll es hinsichtlich der Anforderungen möglich sein die Roboter-Modelle zu animieren. Dazu wurde bereits in der vorangegangenen Projektarbeit ein Algorithmus verwendet bei dem eine Zeitvariable $0 \leq t$ solange um ein Δt erhöht wird, wie $t \leq 1$ ist. Dabei wird bei jedem Iterationsschritt die Transformation des zu animierenden Objekts mit t zwischen der alten und der neuen Transformation interpoliert (Mikos (2025, S. 32)). Mithilfe dieses Algorithmus können einzelne Objekte unabhängig von einander animiert werden. Darauf aufbauend wurde ein neuer Algorithmus entworfen, der in jeder Iteration die Rotation mehrerer Gelenke $j = \{j_0, j_1, \dots, j_n\}$ um die entsprechende Drehachse interpoliert:

Dieser Algorithmus wird in einem separaten Thread ausgeführt. Die dabei verwendete Variable *quality* beeinflusst die Frequenz der Aktualisierungen und damit die

Algorithm 3 Gleichzeitige Interpolation mehrerer Gelenke

```
1:  $t \leftarrow 0$ 
2: while  $t \leq 1$  do
3:    $t_{\text{start}} \leftarrow \text{now}()$ 
4:   for all  $j \in \{j_0, j_1, \dots, j_n\}$  do
5:      $j.\text{transform} \leftarrow \text{interpolate}(j.\text{old\_transform}, j.\text{new\_transform}, t)$ 
6:   end for
7:   for all  $j \in \{j_0, j_1, \dots, j_n\}$  do
8:      $\text{render}(j)$ 
9:   end for
10:   $\text{wait}(0,01/\text{quality})$ 
11:   $t_{\text{end}} \leftarrow \text{now}()$ 
12:   $\Delta t \leftarrow t_{\text{end}} - t_{\text{start}}$ 
13:   $t \leftarrow t + \Delta t$ 
14: end while
15: for all  $j \in \{j_0, j_1, \dots, j_n\}$  do
16:   $j.\text{transform} \leftarrow \text{interpolate}(j.\text{old\_transform}, j.\text{new\_transform}, 1)$ 
17: end for
```

Flüssigkeit der Animation. Ein höherer Wert führt zu einer feineren zeitlichen Auflösung und somit zu einer flüssigeren Bewegung. Auf leistungsschwächeren Geräten kann ein niedrigerer Wert gewählt werden, um die Systemressourcen zu schonen. Dies trägt zur Stabilität des Frameworks bei und gewährleistet, dass Interaktionen wie die Navigation der Kameraperspektive per Maus im Viewport weiterhin problemlos möglich bleiben.

Um die Animation zeitlich konsistenter und realistischer zu gestalten, wird die Interpolationsvariable t nicht um einen festen Wert erhöht, sondern dynamisch anhand der tatsächlich vergangenen Zeit pro Iterationsschritt angepasst. Dazu wird zu Beginn jeder Iteration ein Zeitstempel gesetzt und nach Ausführung der Transformationen sowie einer kurzen Wartezeit die vergangene Zeit gemessen. Der Wert von t wird anschließend um genau diese verstrichene Zeit erhöht. Dieses Verfahren erlaubt eine flüssige und konsistente Animation, auch wenn es zu leichten Schwankungen in der Ausführungsdauer einzelner Iterationen kommt. Da die Interpolation der einzelnen Gelenke auf der gemeinsamen Zeitvariable t beruhen, landen alle Gelenke gleichzeitig in ihrer Zielpose, was der synchronen Point-to-Point Bewegungsplanung entspricht (Weber und Koch (2022, S. 71)).

4.7 grafische Benutzeroberfläche

Das Framework verfügt bereits über die Funktionalität, interaktive Bedienelemente wie Schieberegler oder Checkboxes direkt in ein Jupyter-Notebook einzubetten. Dabei wird auf die Bibliothek ipywidgets zurückgegriffen, die speziell für die Verwendung innerhalb von Jupyter-Umgebungen entwickelt wurde. Für die manuelle Konfiguration der Gelenkwinkel sowie der Bewegung des Endeffektors im lokalen und globalen Koordinatensystem, wie in der Anforderungen beschrieben, wurde ein spezielles Layout entworfen. Dabei wurde sich an der grafischen Benutzeroberfläche von PolyScope5, einer Steuerungssoftware von Universal Robots orientiert. Abbildung 10 zeigt die grafische Benutzeroberfläche von PolyScope5.

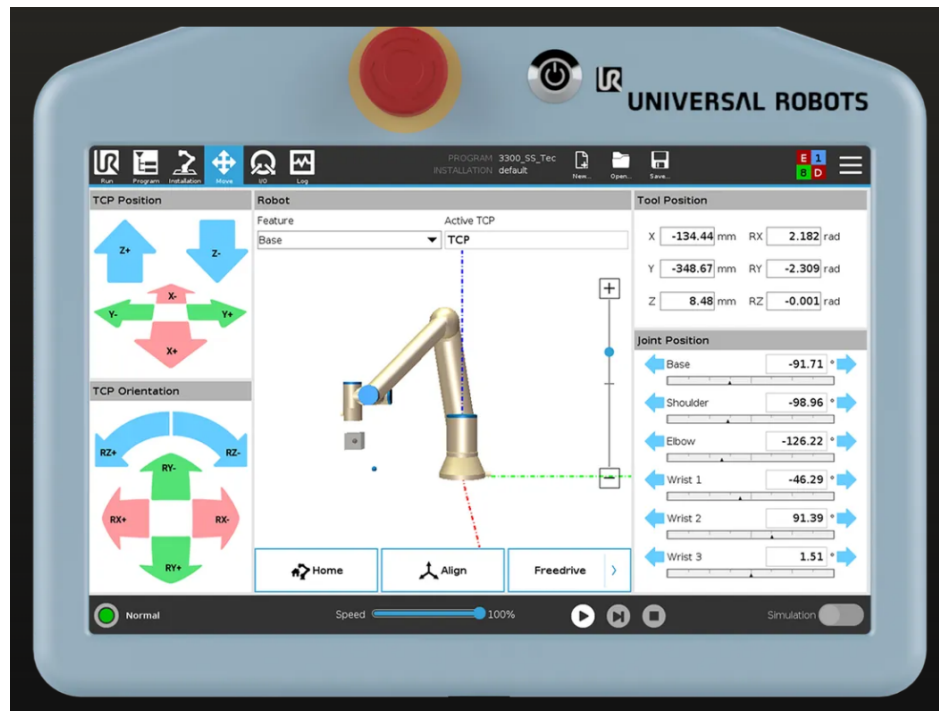


Abbildung 10: Benutzeroberfläche von PolyScope5. Quelle: (Universal Robots (2025))

Das neu entwickelte Layout ermöglicht es sowohl die Gelenkwinkel manuell über Schieberegler einzustellen, als auch die Pose des Endeffektors im lokalen oder globalen Koordinatensystem zu verändern. Neben einzelnen Schiebereglern für jedes Gelenk wurde eine eigene Sektion zur Anzeige und direkten Manipulation der Endeffektor-Pose implementiert. Da während der Entwicklung festgestellt wurde, dass ipywidgets keine Möglichkeit bietet, einen Knopf gedrückt zu halten und damit einen Wert kontinu-

ierlich zu aktualisieren, konnte eine Endeffektorsteuerung wie in Abbildung 10 dargestellt, zunächst nicht vollständig umgesetzt werden. Stattdessen wurde auf Schieberegler zurückgegriffen. Auch hier ergaben sich jedoch Einschränkungen durch die Library: So lässt sich der numerische Wert, der rechts neben dem Schieberegler angezeigt wird, nicht ausblenden. Gerade beim Umschalten auf das lokale Koordinatensystem führt dies zu Problemen hinsichtlich der Benutzerführung. In dieser Darstellungsweise besitzt der Endeffektor stets die Position $(0, 0, 0)$ und Rotation $(0, 0, 0)$ relativ zu sich selbst. Die tatsächliche Transformation wirkt sich zwar korrekt auf die globale Pose aus, die dargestellten Werte im UI gaben jedoch keinen sinnvollen Bezug mehr zur realen Veränderung. In diesem Fall dienten die Schieberegler ausschließlich zur Interaktion, nicht zur Anzeige der tatsächlichen Pose. Eine Möglichkeit zur Ausblendung der Werte wäre daher wünschenswert gewesen. Aufgrund dieser Einschränkungen wurde die Variante mit den Schiebereglern verworfen und ein Workaround erarbeitet, der es ermöglicht Knöpfe gedrückt zu halten. Dazu wurde die Library `ipyevents` genutzt, welche für das Verarbeiten von Maus- und Tastatureingaben innerhalb von Jupyter-Widgets verwendet wird (Project Jupyter contributors (2025)). Die kontinuierliche Interaktion mit einem Knopf wird im Framework durch Auswertung von Mauszuständen realisiert, aus denen ein boolescher Steuerwert abgeleitet wird. Sobald dieser auf `True` gesetzt ist, wird ein Thread gestartet, der eine `while`-Schleife ausführt und so lange aktiv bleibt, bis der Wert wieder auf `False` wechselt. So können im Framework Knöpfe genutzt werden um die Position sowie Orientierung des TCP im Raum zu steuern. Abbildung 11 zeigt das Framework mit viewport und Benutzeroberfläche während der Ausführung innerhalb eines Jupyter-Notebooks

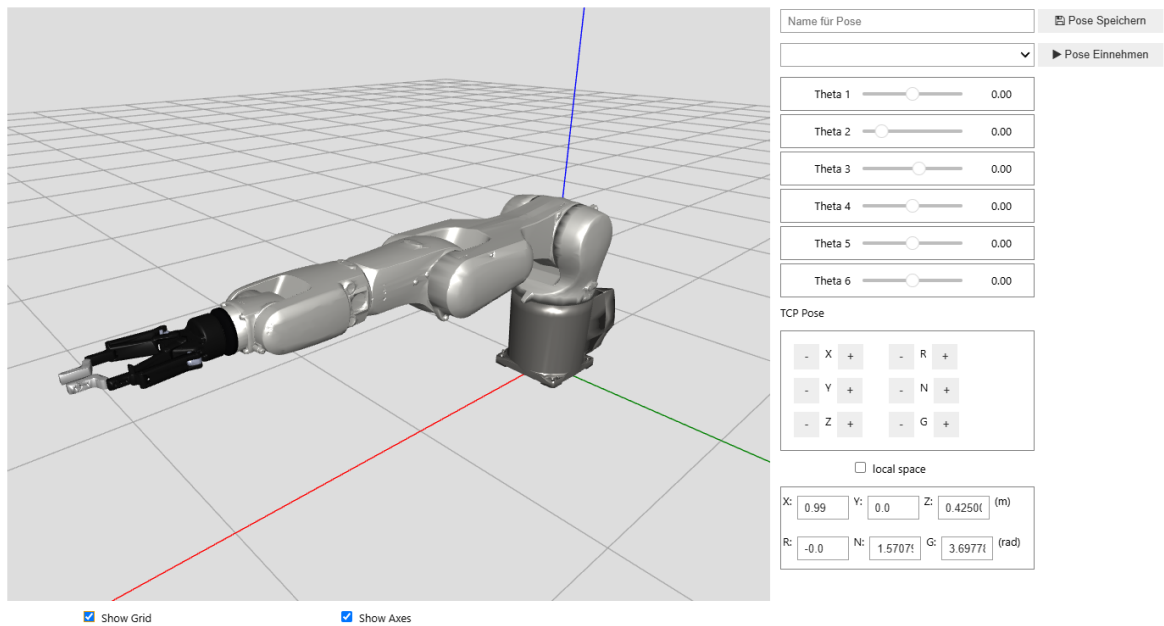


Abbildung 11: Viewport mit grafischer Benutzeroberfläche

Die beiden Felder oben rechts in der Benutzeroberfläche, wie in Abbildung 11 dargestellt, dienen der sogenannten Teach-Funktion. Beim Betätigen des Buttons „Pose speichern“ werden die aktuellen Gelenkwinkel unter dem im danebenstehenden Textfeld angegebenen Namen in einer JSON-Datei gespeichert. Die so gespeicherte Pose ist anschließend über das Dropdown-Menü unter dem vergebenen Namen auswählbar und kann über den Button „Pose Einnehmen“ mithilfe des Animationsalgorithmus wieder angefahren werden.

Darunter befindet sich eine Reihe von Schiebereglern, die dynamisch basierend auf der Anzahl der Gelenke des Roboters erzeugt werden. Jedes Gelenk lässt sich innerhalb der in der URDF-Beschreibung definierten Minimal- und Maximalgrenzen konfigurieren. Diese Werte werden zur Laufzeit eingelesen. Durch Interaktion mit den Reglern wird die jeweilige Joint-Instanz direkt manipuliert, wodurch sich auch alle nachfolgenden Glieder in der kinematischen Kette entsprechend bewegen.

Im Gegensatz zum Animationsalgorithmus erfolgt diese Bewegung nicht zeitlich interpoliert, sondern es wird lediglich die Mesh der jeweiligen Joint-Instanz unmittelbar rotiert. Dadurch entsteht eine intuitive und reaktive Steuerung.

In dem Unteren Bereich namens TCP Pose befinden sich die Knöpfe zur Endeffektorsteuerung in sowie eine die Checkbox um die Steuerung in lokale Koordinaten

umzuschalten. Darunter befinden sich Textfelder um die aktuelle Pose des TCP anzuzeigen, die mithilfe der Vorwärtstransformation berechnet wird.

4.8 Denavit-Hartenberg-Konvention

Um den didaktischen Mehrwert des Frameworks zu erhöhen, wurde die Denavit-Hartenberg-Konvention aufgegriffen und grafisch nachvollziehbar umgesetzt. Hierzu wurde eine eigene Klasse mit dem Namen `DHKinematicModel` entwickelt. Ziel war es, die grafische Darstellung vom mathematischen Berechnungsmodell klar zu trennen, um die Wartbarkeit und Lesbarkeit des Codes zu verbessern. Die Klasse ermöglicht die Erstellung von Homogenen Transformationsmatrizen anhand der jeweiligen DH-Parameter:

- θ_i : Rotationswinkel um die z -Achse
- d_i : Verschiebung entlang der z -Achse
- a_i : Verschiebung entlang der x -Achse
- α_i : Rotationswinkel um die x -Achse

Die Berechnung erfolgt mithilfe der standardisierten DH-Transformationsmatrix, wie sie beispielsweise in (Corke (2023, S. 275)) dargestellt ist:

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Darüber hinaus kapselt die Klasse sowohl die Berechnung der Transformationsmatrix vom globalen Basiskoordinatensystem zur Roboterbasis als auch die Berechnung der Transformation vom letzten Denavit-Hartenberg-Koordinatensystem zum Koordinatensystem des Endeffektors.

Die Transformation von der globalen Szene zur Roboterbasis ist notwendig, um den Roboter beliebig im Raum platzieren zu können. Diese Transformation wird durch folgende homogene Matrix beschrieben:

$$T_{\text{global} \rightarrow \text{base}} = \begin{bmatrix} R & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix} \quad (2)$$

wobei $R \in \mathbb{R}^{3 \times 3}$ eine Rotationsmatrix und $\mathbf{p} \in \mathbb{R}^3$ ein Translationsvektor ist. Die Matrix platziert die Roboterbasis bezüglich des globalen Koordinatensystems und kann der Klasse `DHKinematicModel` im Konstruktor übergeben werden.

Die Berechnung der Transformation vom letzten Denavit-Hartenberg-Koordinatensystem zum Koordinatensystem des Endeffektors erfolgt über das Invertieren der Transformation vom globalen Basiskoordinatensystem zum letzten DH-Koordinatensystem und der nachfolgenden Multiplikation mit der bekannten Pose des Werkzeugs:

$$T_{\text{dh}_n \rightarrow \text{ee}} = T_{\text{global} \rightarrow \text{dh}_n}^{-1} \cdot T_{\text{global} \rightarrow \text{ee}} \quad (3)$$

Dabei bezeichnet $T_{\text{global} \rightarrow \text{dh}_n}$ die gesamte Vorwärtstransformation vom globalen Koordinatensystem bis zum letzten DH-Koordinatensystem, die sich zusammensetzt aus der Matrix $T_{\text{global} \rightarrow \text{base}} \cdot A_1 \cdot A_2 \cdots A_n$. Die resultierende Transformation $T_{\text{dh}_n \rightarrow \text{ee}}$ beschreibt somit die relative Lage des Werkzeugs bezogen auf das letzte DH-Koordinatensystem.

Damit ist die gesamte Vorwärtstransformation nach der Denavit-Hartenberg-Konvention in dieser Klasse gekapselt und modular verfügbar.

Die partiellen Vorwärtstransformationen

$$\mathcal{F}^{(i)} = \left\{ T_{\text{global} \rightarrow \text{base}} \cdot A_1, T_{\text{global} \rightarrow \text{base}} \cdot A_1 A_2, \dots, T_{\text{global} \rightarrow \text{base}} \cdot \prod_{j=1}^i A_j \right\}$$

werden außerhalb der Klasse genutzt, um DH-Koordinatensysteme korrekt im globalen Raum zu platzieren und gemeinsam mit dem Robotermodell visuell darzustellen, wie in Abbildung 12 dargestellt wird.

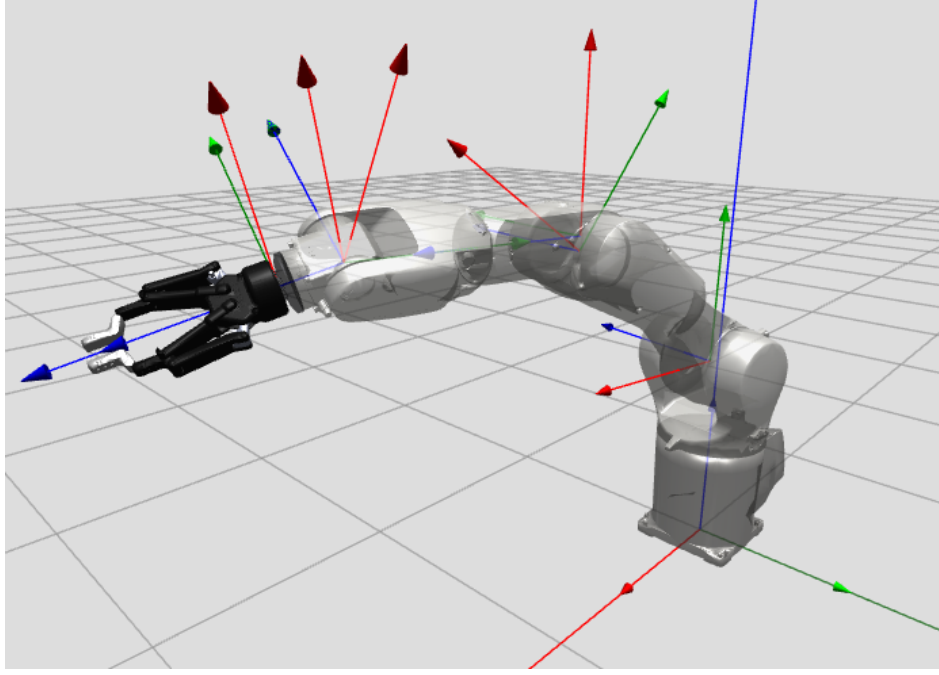


Abbildung 12: Ansicht der 3D-Szene des Frameworks mit dem KR6 R900-2 von KUKA mit Denavit-Hartenberg-Koordinatensystemen

Dazu ist eine Funktion des Frameworks genutzt worden, die bereits im Rahmen der Projektarbeit entwickelt wurde und eine graphische Repräsentation eines Koordinatensystems erstellt. Diese Koordinatensysteme sind aber nicht direkt anhand der Transformationsmatrizen $\mathcal{F}^{(i)}$ transformiert worden. Das hätte den nachteil, dass die Berechnung nach jeder Bewegung des Roboters wiederholt werden müsste. stattdessen wurde der Joint-Klasse eine Variable zur verwaltung des Koordinatensystems K_i hinzugefügt und diese dem Group-Objekt (graphische Repräsentation) des vorherhigen Gelenks j_{i-1} in der kinematischen Kette als Child zugewiesen, sodass die Transformation von j_{i-1} an das Koordinatensystem K_i weitergegeben wird. Da die Transformation von K_i nun relativ zur Transformation des Gelenks j_{i-1} angewendet wird, ist es notwendig die relative Transformation T_{rel} zu berechnen die auf das Koordinatensystem K_i anzuwenden ist um die korrekte Lage zu erhalten. Dazu wird die globale Transformation des Gelenks bzw. des Group-Objekts $T_{j_{i-1}}$ invertiert und mit der globalen Transformation des DH-Koordinatensystems T_{glob_i} multipliziert:

$$T_{rel_i} = T_{j_{i-1}}^{-1} \cdot T_{glob_i}$$

Die relative Transformation T_{rel_i} kann dann einmalig auf K_i angewendet werden, so-

dass dieses anschließend die korrekte Lage im Raum einnimmt und bei der Bewegung des Roboters automatisch durch die Transformationshierarchie mittransformiert wird. Zur besseren Sichtbarkeit der Koordinatensysteme wurde in der Manipulator-Klasse mit `pythreejs` eine Funktion ergänzt, die das Robotermodell transparent erscheinen lässt (Abbildung 12), indem dem Material ein `opacity`-Wert zugewiesen wird. Durch die visuelle Darstellung der DH-Koordinatensysteme kann jede Zwischenkonfiguration entlang der kinematischen Kette nachvollziehbar in der Szene gerendert und analysiert werden, was den abstrakten Berechnungsprozess greifbar und anschaulich macht.

4.9 Rückwärtstransformation (Inverse Kinematik)

Das Framework soll laut den Anforderungen die Möglichkeit bieten, die Pose des Endeffektors (TCP) sowohl über globale als auch lokale Koordinaten zu steuern. Damit sich die Glieder des Roboters entsprechend mitbewegen, müssen aus der Zielpose des TCP die Gelenkkoordinaten $q_0, q_1, \dots, q_n$ im Konfigurationsraum bestimmt werden. Da auch Roboter mit mehr als 6 Gelenken, wie der *LBR iiwa* von KUKA, unterstützt werden sollen, wurde ein numerisches Verfahren auf Basis der Newton-Raphson-Methode implementiert.

Symbolische Vorbereitung

Um die symbolischen Ausdrücke für die Vorwärtstransformation effizient aufzubauen, wird die Mathematik-Library *SymPy* in Kombination mit *NumPy* verwendet. *SymPy* erlaubt die symbolische Definition der Denavit-Hartenberg-Matrizen

$$A_1^{\text{sym}}, A_2^{\text{sym}}, \dots, A_n^{\text{sym}},$$

welche sukzessive zur Gesamttransformation multipliziert werden:

$$A_{\text{ges}}^{\text{sym}} = A_1^{\text{sym}} \cdot A_2^{\text{sym}} \cdot \dots \cdot A_n^{\text{sym}}$$

Die symbolische Transformationsmatrix des TCP ergibt sich somit zu:

$$T_{\text{TCP}}^{\text{sym}} = T_{\text{global} \rightarrow \text{base}} \cdot A_{\text{ges}}^{\text{sym}} \cdot T_{\text{DH}_n \rightarrow \text{EE}}$$

Analytischer Positionsteil

Die Position des Endeffektors $\mathbf{p}_{\text{sym}}$ ist in der vierten Spalte von $T_{\text{TCP}}^{\text{sym}}$ enthalten:

$$T_{\text{TCP}}^{\text{sym}} = \begin{bmatrix} R_{\text{sym}} & \mathbf{p}_{\text{sym}} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

Durch symbolische Ableitung von $\mathbf{p}_{\text{sym}}$ nach den Gelenkwinkeln entsteht die symbolische Jacobi-Matrix für die Position:

$$J_{\text{pos}}^{\text{sym}} = \begin{bmatrix} \frac{\partial p_x}{\partial q_0} & \dots & \frac{\partial p_x}{\partial q_n} \\ \frac{\partial p_y}{\partial q_0} & \dots & \frac{\partial p_y}{\partial q_n} \\ \frac{\partial p_z}{\partial q_0} & \dots & \frac{\partial p_z}{\partial q_n} \end{bmatrix}$$

Diese Matrix wird mit `sympy.lambdify` in eine effiziente NumPy-Funktion überführt, um zur Laufzeit schnell numerische Werte für gegebene Gelenkwinkel berechnen zu können.

Numerischer Rotationsteil

Da die analytische Ableitung der Orientierung (Rotationsmatrix) aufwendig und potenziell instabil ist, wird der entsprechende Jacobi-Anteil numerisch approximiert: Für jede Komponente q_i wird die Orientierung durch kleine Änderungen von q_i variiert und die Änderung als Rotationsvektor $\Delta\boldsymbol{\theta}$ interpretiert:

$$J_{\text{rot}}[:, i] \approx \frac{\boldsymbol{\theta}(R(q_i + \delta)R(q)^{-1})}{\delta}$$

Die daraus resultierende numerische Rotations-Jacobi-Matrix J_{rot} hat ebenfalls Dimension $3 \times n$ und wird mit dem analytischen Positionsteil zu einer vollständigen Jacobi-Matrix zusammengeführt:

$$J_{\text{gesamt}} = \begin{bmatrix} J_{\text{pos}} \\ J_{\text{rot}} \end{bmatrix}$$

Iteratives Lösungsverfahren

Die Lösung der inversen Kinematik erfolgt iterativ. In jedem Schritt wird die Differenz zwischen aktueller und Ziel-Pose (Position und Orientierung) als Fehlervektor $\mathbf{e}$ berechnet:

$$\mathbf{e} = \begin{bmatrix} \mathbf{p}_{\text{ziel}} - \mathbf{p}_{\text{aktuell}} \\ \boldsymbol{\theta}(R_{\text{ziel}} R_{\text{aktuell}}^{-1}) \end{bmatrix}$$

Mithilfe der Pseudoinversen der vollständigen Jacobi-Matrix J_{gesamt} wird dann ein Gelenkwinkel-Inkrement $\Delta \mathbf{q}$ bestimmt:

$$\Delta \mathbf{q} = J_{\text{gesamt}}^+ \cdot \mathbf{e}$$

Der Gelenkwinkelvektor wird entsprechend aktualisiert:

$$\mathbf{q}_{\text{neu}} = \mathbf{q}_{\text{alt}} + \Delta \mathbf{q}$$

Dieser Vorgang wird iteriert, bis der Fehlervektor eine vordefinierte Toleranz unterschreitet oder eine maximale Anzahl an Iterationen erreicht ist. Gelenkwinkelgrenzen können dabei durch Clipping nach jeder Iteration berücksichtigt werden. Es ergibt sich folgender Algorithmus für die implementierung:

Algorithm 4 Iterative Inverse Kinematik mit symbolischem Vorwärtspfad und Gelenkwinkelgrenzen

```

1: Symbolische Vorbereitung:
2:  $A_i^{\text{sym}} \leftarrow$  DH-Matrix für jedes Gelenk  $i$ 
3:  $A_{\text{ges}}^{\text{sym}} \leftarrow A_1^{\text{sym}} \cdot A_2^{\text{sym}} \cdot \dots \cdot A_n^{\text{sym}}$ 
4:  $T_{\text{TCP}}^{\text{sym}} \leftarrow T_{\text{global} \rightarrow \text{base}} \cdot A_{\text{ges}}^{\text{sym}} \cdot T_{\text{DH}_n \rightarrow \text{EE}}$ 
5:  $\mathbf{p}_{\text{sym}} \leftarrow T_{\text{TCP}}^{\text{sym}}[0:3, 3]$ 
6:  $R_{\text{sym}} \leftarrow T_{\text{TCP}}^{\text{sym}}[0:3, 0:3]$ 
7:  $J_{\text{pos}}^{\text{sym}} \leftarrow \frac{\partial \mathbf{p}_{\text{sym}}}{\partial \mathbf{q}}$ 
8:  $fk\_pos, fk\_rot, jacobian\_pos \leftarrow \text{lambdify}(\dots)$ 

9: Iterative Lösung:
10:  $q \leftarrow q_0$  ▷ Initialer Gelenkwinkelvektor
11: for  $i \in \{0, \dots, max\_iters\}$  do
12:    $p_{\text{aktuell}} \leftarrow fk\_pos(q)$ 
13:    $R_{\text{aktuell}} \leftarrow fk\_rot(q)$ 
14:    $e_p \leftarrow p_{\text{ziel}} - p_{\text{aktuell}}$ 
15:    $R_{\text{fehler}} \leftarrow R_{\text{ziel}} \cdot R_{\text{aktuell}}^{-1}$ 
16:    $e_r \leftarrow asRotVec(R_{\text{fehler}})$ 
17:    $e \leftarrow \begin{bmatrix} e_p \\ e_r \end{bmatrix}$  ▷ Gesamter Fehlervektor
18:   if  $\|e\| < tol$  then
19:     return  $q$ 
20:   end if
21:    $J_{\text{pos}} \leftarrow jacobian\_pos(q)$ 
22:    $J_{\text{rot}} \leftarrow \text{zeros}(3, \text{len}(q))$ 
23:   for  $j \in \{0, \dots, \text{len}(q) - 1\}$  do
24:      $\delta q \leftarrow \text{zeros\_like}(q)$ 
25:      $\delta q[j] \leftarrow \delta$ 
26:      $R^+ \leftarrow fk\_rot(q + \delta q)$ 
27:      $R_{\Delta} \leftarrow R^+ \cdot R_{\text{aktuell}}^{-1}$ 
28:      $J_{\text{rot}}[:, j] \leftarrow asRotVec(R_{\Delta}) / \delta$ 
29:   end for
30:    $J_{\text{gesamt}} \leftarrow \begin{bmatrix} J_{\text{pos}} \\ J_{\text{rot}} \end{bmatrix}$ 
31:    $\Delta q \leftarrow J_{\text{gesamt}}^+ \cdot e$ 
32:    $q \leftarrow q + \Delta q$ 
33:   if  $joint\_mins \neq None$  and  $joint\_maxs \neq None$  then
34:      $q \leftarrow clip(q, joint\_mins, joint\_maxs)$ 
35:   end if
36: end for
37: return Fehler: keine Konvergenz innerhalb max_iters

```

Der in Algorithmus 4 dargestellte Ablauf kombiniert die zuvor erläuterten symbolischen und numerischen Schritte zu einem konsistenten IK-Verfahren. Ziel ist es, für eine gegebene Zielpose eine Konfiguration der Gelenkwinkel zu finden, welche diese

Pose möglichst genau erreicht.

Nach Initialisierung des Gelenkwinkelvektors $\mathbf{q}$ wird in jeder Iteration zunächst die aktuelle Pose des TCP berechnet, indem die transformierte symbolische Vorwärtstransformation mit den aktuellen Gelenkwerten ausgewertet wird. Aus der resultierenden Position und Orientierung wird anschließend der Fehlervektor $\mathbf{e}$ gebildet, der die Differenz zur Zielpose beschreibt.

Zur Bestimmung eines Korrekturvorschlags für die Gelenkwinkel wird die zusammengesetzte Jacobi-Matrix J_{gesamt} berechnet. Diese besteht aus einem analytischen Positionsteil sowie einem numerisch approximierten Rotationsanteil. Die dabei verwendeten Funktionen wurden in einem separaten symbolischen Vorbereitungsschritt mit *SymPy* erzeugt und zur Laufzeit mit *NumPy* effizient ausgewertet.

Die Pseudoinverse der Jacobi-Matrix dient zur Berechnung eines Inkrements $\Delta\mathbf{q}$, welches die Gelenkwinkel in Richtung der Zielpose verschiebt. Dabei werden numerische Stabilität und Redundanzbehandlung durch die Pseudoinverse sichergestellt, sodass auch Roboter mit mehr Freiheitsgraden als erforderlich unterstützt werden.

Nach jeder Iteration erfolgt optional eine Begrenzung der Winkel durch Clipping innerhalb der spezifizierten Gelenkgrenzen. Das Verfahren wird wiederholt, bis entweder eine maximale Anzahl an Iterationen erreicht oder die Pose innerhalb einer vorgegebenen Toleranz getroffen wurde.

Die Algorithmusstruktur folgt damit eng dem theoretischen Aufbau:

- symbolische Modellierung der Kinematik über die DH-Parameter,
- Kombination analytischer und numerischer Methoden zur Jacobi-Berechnung,
- schrittweise Konvergenz durch Anwendung der Newton-Raphson-Methode im Gelenkraum.

Nachdem die Rückwärtstransformation implementiert worden ist, wird diese beim verschieben des TCP-Koordinatensystems in der 3D-Szene aufgerufen um die Gelenke und Glieder des Roboters entsprechend mitzubewegen. Da die implementierte Rückwärtstransformation anhand des Newton-Raphson-Verfahrens sehr rechenintensiv ist, wird die symbolische Vorbereitung für jeden Roboter nur einmalig bei der Initialisierung dessen berechnet um Ressourcen zu sparen. Damit konnte die Aktualisierungsrate des Roboters effektiv gesteigert werden. Im gegensatz zum Rendering findet die Berech-

nung der Rückwärtstransformation im Backend statt. Daher steigt die Belastung des Servers bei mehreren Clients die gleichzeitig versuchen die Rückwärtstransformation zu berechnen.

5 Aufbau des Frameworks

5.1 Technischer Aufbau

Das entwickelte Framework ist als eigenständiges Python-Paket organisiert und wurde mithilfe des Packaging-Werkzeugs poetry erstellt. Diese Struktur ermöglicht eine modulare Organisation des Quellcodes, die saubere Trennung von Abhängigkeiten sowie eine einfache Verteilung und Wiederverwendbarkeit des Frameworks (Beuzen (2022, S. 1)). Die Verzeichnisstruktur folgt dem üblichen Aufbau für Python-Pakete mit einem root directory, den entsprechenden Python Modulen, einer pyproject.toml, einer README.md sowie verschiedenen Tests (Beuzen (2022, S. 22)):

```
visualkinematics_v2
├── poetry.lock
├── pyproject.toml
├── README.md
├── assets
│   ├── custom-package-sets
│   └── ros-package-sets
├── src
│   ├── examples
│   │   ├── industrial_robot_example.ipynb
│   │   ├── quader_example.ipynb
│   │   ├── robot_example.ipynb
│   │   └── rotations_example.ipynb
│   └── visualkinematics_v2
│       ├── __init__.py
│       ├── manager.py
│       ├── manipulator.py
│       ├── show.py
│       ├── util.py
│       └── _package_data
└── tests
```

Abbildung 13: Verzeichnisstruktur des Packets visualkinematics_v2

Durch diese Paketstruktur kann das Framework problemlos in andere Projekte integriert, installiert oder erweitert werden. Vorgefertigte Beispielnotebooks befinden sich im Ordner `examples` und veranschaulichen die Funktionsweise des Frameworks. Darunter befinden sich die Notebooks `quader_example.ipynb`, `robot_example.ipynb` sowie `rotations_example.ipynb`, die bereits im Rahmen der Projektarbeit entwickelt worden sind und die Funktionsweise von Transformationen von einfachen Körpern sowie von mobilen Robotern in Form von Übungsaufgaben demonstrieren. Das Beispielnotebook `industrial_robot_example.ipynb` stellt die Nutzung des Frameworks zur Simulation von Industrierobotern dar. Alle Beispiel-Notebooks greifen direkt auf den Quellcode des Pakets zu.

Der Quellcode befindet sich im Verzeichnis `visualkinematics_v2/src/visualkinematics_v2`. Dieser ist in einzelne Module gegliedert, die jeweils einer bestimmten Verantwortlichkeit zugeordnet sind. Ein Überblick über die zentralen Module und deren Beziehungen ist in Abbildung 17 (siehe Anhang) in Form eines Packetdiagramms dargestellt. Es wurde darauf geachtet, dass zwischen den Modulen keine bidirektionalen oder zyklischen Abhängigkeiten bestehen.

5.2 Softwarearchitektur Model-View-Controller

Der Quellcode des Pakets verteilt sich auf insgesamt 5 Module: `util.py`, `manipulator.py`, `manager.py` und `show.py`. Das Zusammenspiel dieser Module folgt der Softwarearchitektur des Model-View-Controller (MVC) Paradigmas, welches sich für grafische Echtzeitanwendungen eignet. Ziel dieser Trennung ist eine modulare Struktur, die eine klare Abgrenzung zwischen Daten (Model), Präsentation (View) und Interaktion/-Steuerung (Controller) ermöglicht. Diese Struktur erleichtert die Erweiterbarkeit und Wartbarkeit des Frameworks (Lee (2024, S. 1000)).

5.2.1 Model-Komponente

Das Modul `util.py` stellt eine zustandslose Sammlung fachspezifischer Kern- und Hilfsfunktionen dar und lässt sich am ehesten der Model-Komponente zuordnen. Es wird von allen anderen Modulen importiert und genutzt. Die Model-Komponente wird vor allem durch die Module `manipulator` und `manager` realisiert. Sie bildet die strukturelle und funktionale Grundlage des Frameworks und kapselt die kinematische Struktur des

Roboters sowie dessen aktuellen Zustand.

Das Modul manipulator enthält zentrale Klassen wie:

- **Manipulator** – verwaltet den Gesamtaufbau des Roboters, einschließlich aller Glieder und Gelenke.
- **Link** – repräsentiert ein einzelnes physisches Glied der kinematischen Kette und enthält Informationen zur Geometrie und lokalen Transformation.
- **Joint** – modelliert ein Gelenk zwischen zwei Gliedern inklusive Bewegungsbereich und aktuellem Winkel.
- **Kinematic_Chain_Element** – Die Oberklasse von Link und Joint und verwaltet die Children und den Parent die sich aus der kinematischen Kette ergeben.
- **DHKinematicModel** – kapselt die mathematische Berechnung der Vorwärts- und Rückwärtstransformation auf Basis der Denavit-Hartenberg-Parameter.

Innerhalb dieser Klassen werden alle relevanten Informationen zur Beschreibung und Steuerung eines Industrieroboters zusammengeführt. Dazu zählen die Hierarchie der Glieder, die Relativtransformationen entlang der kinematischen Kette sowie der aktuelle Gelenkzustand (z. B. Positionen und Winkel der Achsen). Diese Daten bilden die Grundlage für die spätere grafische Darstellung und Animation. Die Denavit-Hartenberg-Konvention sowie die darauf basierende Berechnung der Vorwärts- und Rückwärtstransformation wurden in die Klasse DHKinematicModel ausgelagert. Dadurch wird eine saubere Trennung der Zuständigkeiten erreicht und gleichzeitig die Wartbarkeit sowie die Lesbarkeit des Codes verbessert.

Innerhalb der Model-Komponente wird die Datenhaltung und somit der Zugriff auf den persistenten Speicher in das Modul manager entkoppelt. Hier befinden sich Funktionen zum Laden und Speichern von 3D-Modellen sowie URDF-Dateien. Darüber hinaus erzeugt das Modul manager eine Datei namens config.toml in einem standardisierten Verzeichnis für benutzerspezifische Konfigurationsdaten. Der genaue Speicherort ist betriebssystemabhängig. Unter Windows befindet sich dieser beispielsweise unter C:/Users/<Benutzername>/AppData/Local/visualkinematics_v2, während auf Linux-Systemen typischerweise das Verzeichnis /.config/visualkinematics_v2 verwendet wird. Dieses Vorgehen entspricht den Konventionen der jeweiligen Plattformen und

ermöglicht eine konsistente sowie benutzerspezifische Speicherung von Anwendungseinstellungen. In der Datei `config.toml` werden die Speicherorte für Ressourcen und Assets definiert. Der Benutzer hat die Möglichkeit, diese Pfade bei Bedarf manuell anzupassen, um beispielsweise benutzerdefinierte Verzeichnisse zu verwenden. In den konfigurierten Verzeichnissen können benutzereigene 3D-Modelle und URDF-Beschreibungen abgelegt werden. Diese stehen dem Framework anschließend zur Verfügung und können direkt geladen werden. Damit das bereitgestellte Beispiel-Notebook `industrial_robot_example.ipynb` ohne weitere Anpassungen („out of the box“) lauffähig ist, werden mit dem Paket im Verzeichnis `_package_data` bereits einige 3D-Modelle und URDF-Beschreibungen mitgeliefert.

5.2.2 View-Komponente

Die View ist für die Darstellung der Szene im Jupyter Notebook zuständig und wird durch das Modul `show` dargestellt. Sie verwendet `pythreejs` sowie `ipywidgets` zur visuellen Repräsentation und enthält folgende Klassen und Funktionen:

- **Environment:** erzeugt die 3D-Szene mit Licht, Kamera, Koordinatensystem usw.
- `_ipython_display_()`: bindet die Szene beim aufrufen von `display()` als interaktives Widget in das Notebook ein.
- **Inspector_View:** enthält GUI-Elemente wie Schieberegler und Knöpfe für die Steuerung und Konfiguration eines Roboters.
- **Gizmo_Controls_View:** enthält GUI-Elemente für die Translation, Rotation und Skalierung eines beliebigen Körpers.

5.2.3 Controller-Komponente

Der Controller bildet die Schnittstelle zwischen der Benutzerinteraktion und dem zugrunde liegenden Datenmodell. Er verknüpft die GUI-Elemente der View mit den Funktionen des Models und steuert das Verhalten des Frameworks in Abhängigkeit von Benutzereingaben. Dabei kommt das Modul `ipyevents` zum Einsatz, das eine Ereignisverarbeitung in Jupyter-Notebooks ermöglicht. Der Controller ist wie die View-Komponente ebenfalls im Modul `show` implementiert und wird dort in den folgenden Klassen gekapselt:

- **Inspector_Controller**: Verbindet die GUI-Elemente von Inspector_View mit Funktionen und speichert Zustände wie mouse_down.
- **Gizmo_Controls_Controller**: Verbindet die GUI-Elemente der Gizmo_Controls_View mit Funktionen.

Zwischen der View und dem Controller sowie zwischen dem Model und dem Controller besteht eine lose Kopplung. Die Klasse Inspector_Controller besitzt eine Referenz auf die Klasse Manipulator (Model), wodurch eine einseitige Assoziation zwischen Controller und Model besteht. Die Klasse Inspector_View kennt die Manipulator-Instanz ebenfalls, allerdings ausschließlich lesend, um beispielsweise Initialwerte für GUI-Elemente wie Schieberegler setzen zu können. Eine detaillierte Darstellung der beteiligten Module, Klassen und ihrer Beziehungen befindet sich in Abbildung 16 (siehe Anhang) in Form eines Klassendiagramms.

6 Deployment des Frameworks

Das Framework wird auf einem zentralen Server (siehe Tabelle 7) des Instituts für die Digitalisierung von Arbeits- und Lebenswelten (IDiAL) der Fachhochschule Dortmund gehostet. Ziel ist es, einen benutzerdefinierten Python-Kernel bereitzustellen, der allen Benutzern zentral zur Verfügung steht und auf das Framework samt Abhängigkeiten zugreifen kann. Die Paketverwaltung erfolgt mithilfe von Poetry Version 2.1.2, um eine reproduzierbare und isolierte Entwicklungsumgebung sicherzustellen. Das Zielsystem stellte standardmäßig Python in Version 3.8.10 bereit, welche jedoch nicht kompatibel mit allen Abhängigkeiten des Projekts war. Daher wurde Python 3.10 von python.org heruntergeladen und manuell im Home-Verzeichnisses des Benutzers philipp kompiliert und installiert. Dabei wurde die Systeminstallation Python 3.8.10 nicht verändern oder gelöscht:

```
./configure --prefix=$HOME/python-3.10
make
make install
export PATH="$HOME/python-3.10/bin:$PATH"
source ~/.bashrc
```

Anschließend wurde das Repository von GitHub geklont. Da es sich um ein privates Repository handelt wurde ein SSH-Schlüssel generiert und dieser bei GitHub eingerichtet. Danach wurde das Projekt eingerichtet und anschließend in das Verzeichnis /opt verschoben um eine zentrale Nutzung zu ermöglichen:

```
git clone git@github.com:PSoldOut/visualkinematics_v2.git
cd visualkinematics_v2
poetry install
sudo mv ../visualkinematics_v2 /opt
```

Die dadurch erstellte virtuelle Umgebung wurde als globaler Kernel für alle JupyterHub-Nutzer registriert. Dazu wurde folgender Befehl genutzt:

```
'sudo $(poetry run which python) -m ipykernel install
--name=visualkinematics_v2
--display-name "VisualKinematics v2"
--prefix=/usr/local'
```

Dies installiert den Kernel systemweit in /usr/local/share/jupyter/kernels/. Danach ist der Kernel im JupyterHub sichtbar und kann von allen Nutzern verwendet werden. Der Zugriff auf das Poetry-Projekt und dessen Bibliotheken ist gegeben. Nutzer können nun direkt mit dem Framework in Jupyter arbeiten. Damit die Schieberegler und Knöpfe des Frameworks ordnungsgemäß funktionieren, musste ipyevents und die zugehörige Labextension auf dem Host installiert werden:

```
sudo /opt/tljh/user/bin/python3 -m pip install ipyevents
sudo /opt/tljh/user/bin/jupyter labextension install ipyevents
```

7 Evaluation

7.1 Stresstest zur Ressourcen- und Performance-Analyse

Um den Ressourcenverbrauch und die Performance des Frameworks unter Mehrfachverbindungen zu analysieren, wurde ein Stresstest durchgeführt. Im Fokus standen dabei insbesondere die Ausführung von Animationen sowie die Berechnung der Vorwärts- und Rückwärtstransformationen während des Betriebs, da das Funktionen sind die im realen Betrieb durchgehend aufgerufen werden. Das Laden der Meshes wurde in diesem Test nicht berücksichtigt, da es lediglich einmalig beim Start des Frameworks erfolgt.

7.1.1 Testaufbau

Der Stresstest wurde am 25.07.2025 und 06.08.2025 im Computerraum C.1.30 des Fachbereichs Informatik der Fachhochschule Dortmund durchgeführt. Dabei kam ein Laptop der Marke HP (System B LT, siehe Tabelle 3) als Server zum Einsatz. Als Clients wurden acht Workstations der Marke Dell (System C WS, siehe Tabelle 4) verwendet. Für die Netzwerkkommunikation wurde ein Access Point der Marke TP-Link (System D AP, siehe Tabelle 5) mit einem Frequenzbereich von 2.4GHz eingesetzt. Dieser wurde ausschließlich für den Test verwendet, sodass keine zusätzliche Auslastung des WLANs vorlag. Server und Clients waren alle gleichzeitig über WLAN mit dem Access Point verbunden. Für die Kommunikation zwischen Clients und Server wurde JupyterLab verwendet. Jeder Client kommunizierte mit genau einem eigenen Kernel-Prozess über den Google Chrome Webbrowser. Somit bestand immer eine 1:1-Zuordnung zwischen Kernel und Client.



Abbildung 14: Testaufbau der Ressourcen- und Performance-Analyse

7.1.2 Messmethoden

Um bei der Durchführung des Tests die Aktualisierungsrate einer Animation messen zu können wurde temporär eine Anzeige im UI des Frameworks hinzugefügt. Diese zeigte jeweils die aktuell berechneten Schritte pro Sekunde (SPS) an. Zu diesem Zweck wurde in der Animationsschleife eine entsprechende Funktion integriert, die die Anzeige bei jedem Zyklus aktualisierte. Die Bildrate (FPS) der Animation im Browser wurde über die „FPS“-Extension aus dem Chrome Web Store gemessen. Die Unterscheidung zwischen FPS (Frames per Second) und SPS (Steps per Second) ist notwendig, da eine flüssige Bildrate im Browser allein nicht garantiert, dass die visuellen Schritte der Animation performant genug sind. Auch bei stabilen 60 FPS kann eine zu langsame Verarbeitung der einzelnen Schritte zu sichtbaren Rucklern führen. Zur Messung der Systemauslastung auf dem Server (System B LT) wurde alle 5 Sekunden die Ausgabe des Kommandozeilenprogramms `top` in eine Datei geschrieben. Um die Auslastungswerte auf den Clients zu messen wurde der Taskmanager von Windows verwendet.

7.1.3 Durchführung

Zur Durchführung des Tests wurden am 25.07.2025 zunächst auf dem System B LT acht unabhängige JupyterLab-Instanzen gestartet, wobei jede Instanz mit einem eigenen Kernel-Prozess verbunden war. Parallel dazu wurde auf acht Clients jeweils ein Browser-Tab in Google Chrome geöffnet, der mit genau einer dieser Kernel-Instanzen kommunizierte. Dann wurde auf jedem Client eine Endlosschleife ausgeführt, in der kontinuierlich eine Rückwärtstransformation berechnet und eine Animation ausgeführt wurde. Die Rückwärtstransformation beinhaltet auch die Berechnung der Vorwärtstransformation. Währenddessen wurde die Systemauslastung auf dem Server über einen Zeitraum von 3 Minuten gemessen, und zwar entsprechend der in Kapitel 7.1.2 beschriebenen Methode. gleichzeitig wurden auf einem der Clients zusätzlich die Auslastungswerte aus dem Taskmanager mithilfe des Snipping Tools dokumentiert. Nach Abschluss der Messung mit 8 Clients wurde das gleiche Verfahren mit nur 4 Clients und 4 JupyterLab-Instanzen wiederholt. Aus zeitlichen Gründen musste an dieser Stelle der Test unterbrochen und am 06.08.2025 fortgesetzt werden. Zur Fortsetzung des Tests wurde auf dem Server eine JupyterLab-Instanz gestartet mit der ein Client über den Browser kommunizierte und die Endlosschleife ausführte. Währenddessen wurde die Bildrate (FPS) der Animation im Browser für die restliche Dauer des Tests in der FPS Extension beobachtet. Gleichzeitig wurde die SPS der Animation über die temporär hinzugefügte Anzeige im UI des Frameworks bestimmt. Dafür wurden nach jeweils zufälligen Zeitintervallen insgesamt 10 SPS-Werte auf ganze Zahlen gerundet und manuell notiert. Im weiteren Verlauf wurden auf dem Server (System B LT) 4 und dann 8 JupyterLab-Instanzen gestartet mit der jeweils ein Client kommunizierte und die Endlosschleife ausführte. Dabei wurden die FPS und SPS pro Verbindung auf die selbe Weise erhoben.

7.1.4 Ergebnisse

Die top-Ausgabe, welche dieser Arbeit als `top_out8clients.txt` und `top_out4clients.txt` im Ordner `perfTest` beiliegt, zeigt für 8 Instanzen eine durchschnittliche CPU-Auslastung von 98,8% davon 92,33% im User Space wo die JupyterLab-Instanzen ausgeführt worden sind. Der Load Average lag bei 12,15. Die gesamte RES-Speichernutzung der 8 JupyterLab-Instanzen lag im Durchschnitt bei 3483.22 MiB. Für 4 Instanzen lag eine durchschnittliche CPU-Auslastung von 98,9% davon 69,57% im User Space. Der Load

Average lag hier bei 8,07. Die gesamte RES-Speichernutzung bei 1621,85 MiB

Inst.	CPU Auslastung (%)	User Space (%)	Load Average	RES (MiB)
8	98,8	92,3	12,15	3483,22
4	98,9	69,6	8,07	1621,85

Tabelle 2: Systemauslastung bei 4 und 8 JupyterLab-Instanzen

Die Aktualisierungsrate der Animation (siehe Anhang) unterlag deutlichen Schwankungen. Der Mittelwert lag bei 216,7 SPS bei 8 Clients, 79,25 SPS bei 4 Clients und 40,9 SPS bei einem einzelnen Client. Abbildung 15 zeigt ein Boxplot-Diagramm der erhobenen Messwerte.

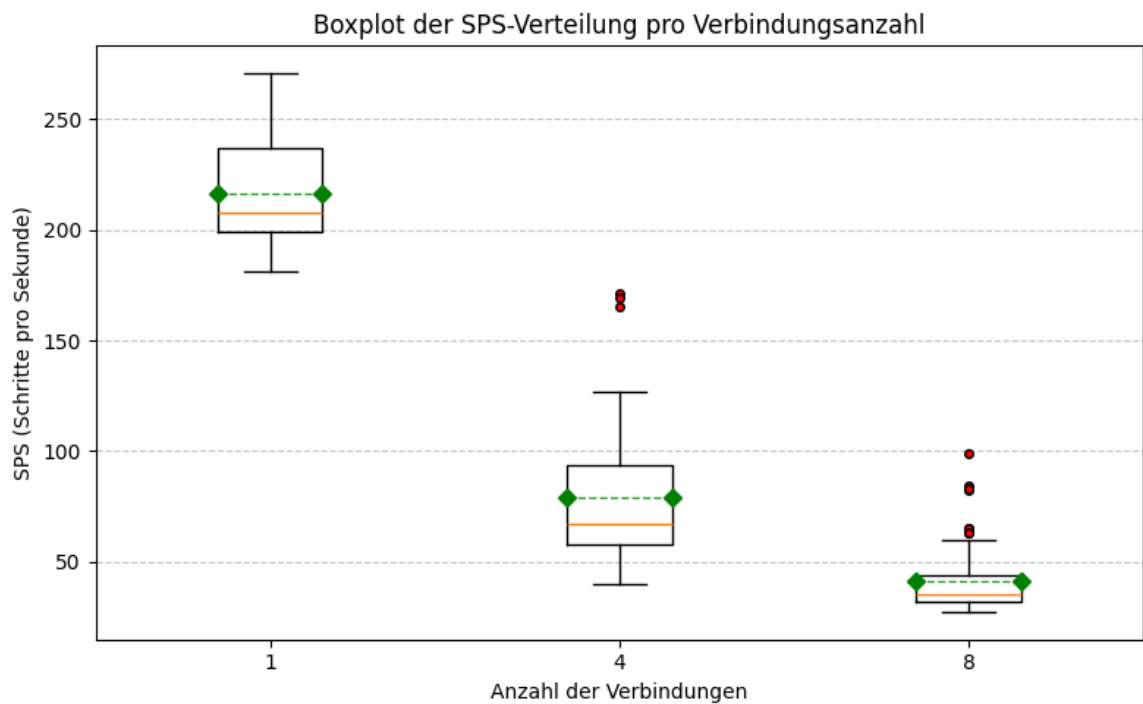


Abbildung 15: Verteilung der gemessenen SPS-Werte je nach Anzahl der Clients

Die gestrichelte grüne Linie markiert den jeweiligen Mittelwert. Die orangefarbene Linie kennzeichnet den Median. Die sogenannten Whisker (schwarze Linien mit Endstrichen) zeigen die Spannweite der Daten unter Berücksichtigung der Standardabweichung. Ausreißer werden durch die roten Punkte gekennzeichnet. Trotz der hohen CPU-Last auf dem Server (System B LT) lief die Animation auf allen Clients weitestgehend flüssig ab. Die Bildrate in den Browserfenstern blieb bei allen Clients über die

gesamte Dauer des Tests stabil zwischen 59 und 60 FPS. Auf einem der Clients zeigte der Windows-Taskmanager während des Tests eine CPU-Auslastung von 12 %, eine GPU-Auslastung von 18 % sowie eine RAM-Nutzung von 7,2 GB.

7.1.5 Interpretation

Die Messergebnisse verdeutlichen, dass bereits bei 4 gleichzeitig verbundenen Clients eine nahezu vollständige Auslastung der CPU des Testsystems (System B LT) erreicht wurde. Bei 8 Instanzen stieg die CPU-Auslastung im User Space von 69.6% auf 92.3%, was auf eine erhebliche Steigerung der Rechenlast durch die zusätzlichen JupyterLab-Instanzen hinweist.

Der Load Average überschritt in beiden Fällen deutlich die Anzahl verfügbarer logischer CPU-Kerne, was bedeutet, dass viele Prozesse gleichzeitig auf CPU-Zeit warteten. Besonders bei 8 Instanzen mit einem Load Average von 12.15 ist eine deutliche Systemüberlastung erkennbar.

Der Arbeitsspeicherverbrauch stieg bei 8 Instanzen auf mehr als das Doppelte im Vergleich zu 4 Instanzen. Dies ist erwartbar, da jede Instanz Speicher benötigt, allerdings zeigt sich auch ein zusätzlicher Overhead pro Instanz.

Diese Ergebnisse zeigen, dass das verwendete Testsystem (System B LT) an seine Kapazitätsgrenzen gelangt. Da es sich hierbei nicht um das geplante produktive System handelt, können die Werte nicht unmittelbar auf die spätere Einsatzumgebung übertragen werden. Sie zeigen jedoch, dass bei Mehrfachnutzung eine ausreichend leistungsfähige Serverhardware erforderlich ist.

Die Tatsache, dass die Animation auf den Clients dennoch Flüssig lief bestätigt, dass die Berechnungen für das Rendering im Frontend durchgeführt werden und das Framework somit trotz Überlastung eine stabile Nutzererfahrung ermöglicht. Dies erklärt auch die leicht erhöhte GPU Auslastung auf dem Client.

8 Anhang

Komponente	Spezifikation
Gerätetyp	Laptop
Hersteller	HP
Modell	Pavilion Laptop 14-bf0xx
Prozessor (CPU)	Intel® Core™ i5-7200U CPU mit 2,50 GHz, 2 Kerne, 4 Threads
Arbeitsspeicher (RAM)	8 GB (1x 8 GB, DDR4, 2133 MHz, SODIMM, SK Hynix)
Grafikeinheit (GPU)	Intel HD Graphics 620 (integriert) NVIDIA GeForce 940MX (dediziert)
Massenspeicher	Samsung SSD 256 GB
Betriebssystem	Linux Mint 21.3 „Virginia“ (64-bit)

Tabelle 3: Technische Spezifikationen des Testsystems „System B LT“

Komponente	Spezifikation
Gerätetyp	Workstation (Micro Form Factor)
Hersteller	Dell
Modell	OptiPlex 7000 Micro
Prozessor (CPU)	Intel® Core™ i5-12500T CPU mit 2,00–4,60 GHz, 6 Kerne, 12 logische Prozessoren
Arbeitsspeicher (RAM)	16 GB DDR4 (1× 16 GB, 3200 MHz, SODIMM)
Grafikeinheit (GPU)	Integrierte Intel UHD Graphics 770
Massenspeicher	256 GB NVMe SSD (PCIe 3.0 x4, M.2)
Betriebssystem	Windows 11 Pro (64-bit), Version 22H2 (Build 22621)

Tabelle 4: Technische Spezifikationen des Testsystems „System C WS“

Komponente	Spezifikation
Gerätetyp	Access Point (Indoor)
Hersteller	TP-Link
Modell	TL-WA801N
WLAN-Standard	IEEE 802.11n (2.4 GHz)
Maximale Übertragungsrate	300 Mbit/s
Ethernet-Anschluss	1× Fast Ethernet (10/100 Mbit/s)
Antennen	2× externe, abnehmbare Antennen
Betriebssystem	Eigenständige Firmware (Webinterface konfigurierbar)

Tabelle 5: Technische Spezifikationen des Testsystems „System D AP“

Komponente	Spezifikation
Gerätetyp	Desktop
Hersteller	Eigenbau (ASUS-Mainboard, OEM-Konfiguration)
Prozessor (CPU)	Intel® Core™ i7-9700K CPU mit 3,60 GHz, 8 Kerne, 8 logische Prozessoren
Arbeitsspeicher (RAM)	16 GB (2x 8 GB, 3000 MHz, DIMM)
Grafikeinheit (GPU)	NVIDIA GeForce RTX 2070
Massenspeicher	Samsung SSD 970 EVO Plus 500 GB
Betriebssystem	Windows 10 Home (64-bit), Version 10.0.19045 Build 19045

Tabelle 6: Technische Spezifikationen des Testsystems „System A DT“

Komponente	Spezifikation
Gerätetyp	Virtuelle Maschine (KVM)
Prozessor (CPU)	AMD EPYC Processor, 3,0 GHz, 8 Kerne, 8 Threads
Arbeitsspeicher (RAM)	16 GB
Cache	L1: 256 KiB, L2: 4 MiB, L3: 64 MiB
Betriebssystem	Ubuntu 20.04.6 LTS (64-bit)
Kernel-Version	Linux 5.4.0-216-generic
Architektur	x86_64 (64-bit)
Hypervisor	KVM
Virtualisierungstyp	Vollvirtualisierung

Tabelle 7: Technische Spezifikationen des Zielservers (JupyterHub-VM)

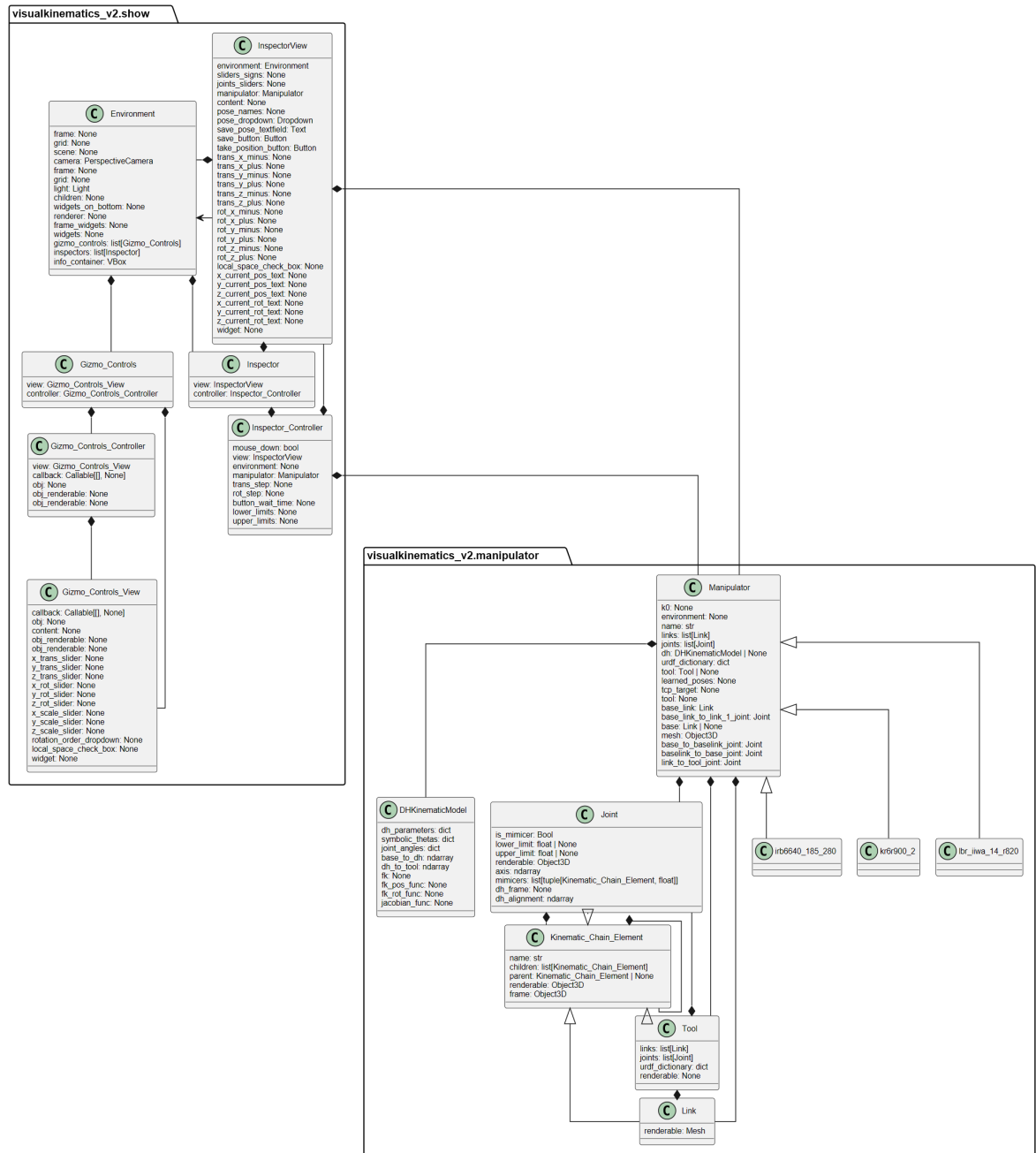


Abbildung 16: UML-Klassendiagramm des Frameworks generiert mit dem Tool py2puml

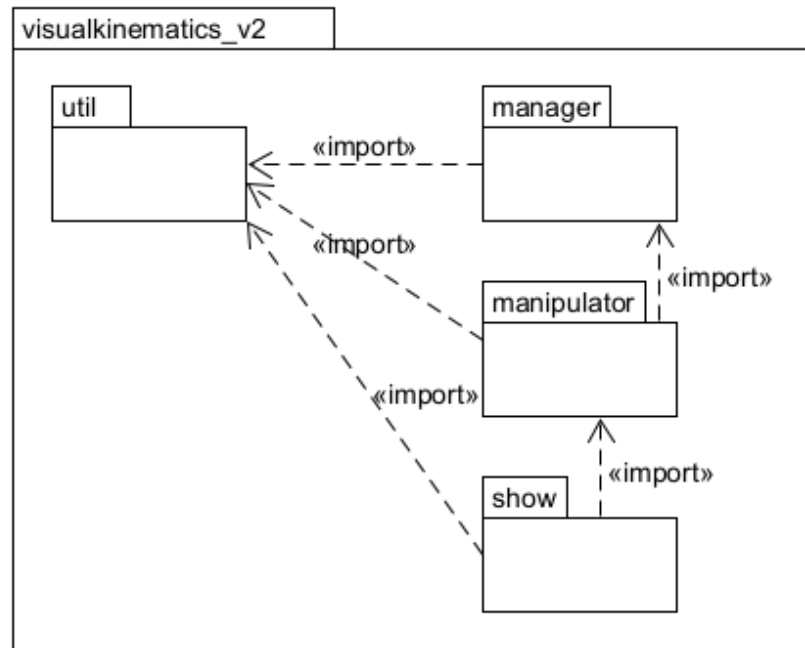


Abbildung 17: UML-Packagediagramm der Module

Literatur

- Beuzen, Tomas (2022). *Python Packages*. CRC Press. ISBN: 9781003189251.
- Corke, Peter (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python*. eng. Third edition. Bd. 146. Springer Tracts in Advanced Robotics. Cham: Springer International Publishing AG. ISBN: 3031064682.
- GrabCAD Contributors (2025). *About GrabCAD*. URL: <https://resources.grabcad.com/company/> (besucht am 10.07.2025).
- Hallmann, Martin, Stefan Goetz und Benjamin Schleich (2019). „Mapping of GD(and)T information and PMI between 3D product models in the STEP and STL format“. In: *Computer-Aided Design* 115, S. 293–306. ISSN: 0010-4485. DOI: <https://doi.org/10.1016/j.cad.2019.06.006>. URL: <https://www.sciencedirect.com/science/article/pii/S0010448518305359>.
- Heins, Adam (2025). *reference xacrodoc 0.7.0 documentation*. URL: <https://xacrodoc.readthedocs.io/en/latest/reference.html#> (besucht am 14.07.2025).
- KUKA Contributors (2025). *Feedback Erfolgsmeldung*. URL: <https://www.kuka.com/de-de/feedback/erfolgsmeldung> (besucht am 02.07.2025).
- Lee, Newton (2024). „MVC Architecture“. eng. In: *Encyclopedia of Computer Graphics and Games*. Cham: Springer International Publishing, S. 1223–1223. ISBN: 3031231597.
- Mikos, Philipp (2025). „Visualisierung kinematischer Kennwerte“. In: *Projektarbeit, Fachhochschule Dortmund, Studiengang BA Informatik 1*.
- Mileff, Péter (2024). „PER-PIXEL LIGHTING FOR MULTI-OBJECT MODELS“. In: *Production Systems and Information Engineering* 12.3, S. 42–59.
- NumPy Contributors (2025). *NumPy .npy and .npz file format*. URL: <https://numpy.org/devdocs/reference/generated/numpy.savez.html> (besucht am 11.07.2025).
- Open Source Robotics Foundation (2023). *Packages - ROS Wiki*. URL: <https://wiki.ros.org/Packages> (besucht am 10.07.2025).
- Project Jupyter contributors (2025). *ipyevents pypi*. URL: <https://pypi.org/project/ipyevents/> (besucht am 26.07.2025).
- PTC (2025). *why Onshape?* URL: <https://www.onshape.com/en/why-onshape> (besucht am 11.07.2025).

- pythreejs Contributors (2025a). *API reference - pythreejs 2.4.1 documentation*. URL: <https://pythreejs.readthedocs.io/en/stable/api/index.html> (besucht am 10.07.2025).
- (2025b). *Geometry types*. URL: <https://pythreejs.readthedocs.io/en/stable/examples/Geometries.html> (besucht am 10.07.2025).
- Reinhart, Gunther, Alejandro Magaña Flores und Carola Zwicker (2018). *Industrieroboter*. ger. 1. Aufl. Würzburg: Vogel Communications Group. ISBN: 9783834362360.
- ROS-Industrial Contributors (2025a). *Repository search result*. URL: <https://github.com/search?q=topic%3Amoveit+org%3Aros-industrial+fork%3Atrue&type=repositories> (besucht am 10.07.2025).
- (2025b). *ROS-Industrial*. URL: <https://github.com/ros-industrial> (besucht am 10.07.2025).
- Three.js Contributors (2025). *MeshStandardMaterial - three.js docs*. URL: <https://threejs.org/docs/#api/en/materials/MeshStandardMaterial> (besucht am 13.07.2025).
- trimesh Contributors (2025). *trimesh*. Version 3.2.0. URL: <https://trimesh.org/> (besucht am 01.07.2025).
- UltiMaker (2025). *Thingiverse - Digital Designs for Physical Objects*. URL: <https://www.thingiverse.com/about> (besucht am 10.07.2025).
- Universal Robots (2025). *PolyScope 5*. URL: <https://www.universal-robots.com/products/polyscope-5/> (besucht am 15.07.2025).
- Weber, Wolfgang und Heiko Koch (2022). *Industrieroboter*. 5., aktualisierte und erweiterte Auflage. München: Carl Hanser Verlag GmbH & Co. KG. DOI: 10.3139/9783446468702. eprint: <https://www.hanser-elibrary.com/doi/pdf/10.3139/9783446468702>. URL: <https://www.hanser-elibrary.com/doi/abs/10.3139/9783446468702>.
- Weingartshofer, T., M. Schwegel, C. Hartl-Nesic, T. Glück und A. Kugi (2019). „Collaborative Synchronization of a 7-Axis Robot“. In: *IFAC-PapersOnLine* 52.15. 8th IFAC Symposium on Mechatronic Systems MECHATRONICS 2019, S. 507–512. ISSN: 2405-8963. DOI: <https://doi.org/10.1016/j.ifacol.2019.11.726>. URL: <https://www.sciencedirect.com/science/article/pii/S2405896319317185>.